



UNIT 1

Introduction to DevOps & DevOps Lifecycle

Learning Objectives

After completing this unit, students will be able to:

- Understand the concept of DevOps.
 - Explain the goals and benefits of DevOps.
 - Describe the DevOps lifecycle.
 - Identify the roles and responsibilities of a DevOps Engineer.
 - Explain the principles of DevOps.
 - Compare Agile and DevOps.
 - Understand Infrastructure as Code (IaC).
-

1.1 Introduction to DevOps

What is DevOps?

DevOps is a combination of two words:

- **Development (Dev)** – Software development team
- **Operations (Ops)** – IT operations team

DevOps is a culture, set of practices, and collection of tools that improve collaboration between software developers and IT operations teams. It automates software development, testing, deployment, and monitoring to deliver software faster and more reliably.

Definition

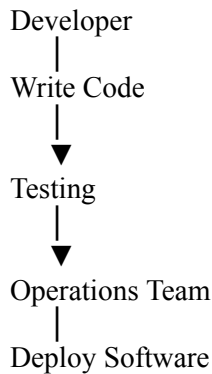
DevOps is a software engineering methodology that integrates software development and IT operations through automation, collaboration, and continuous delivery to produce high-quality software efficiently.



Why Was DevOps introduced?

Traditionally, development and operations teams worked independently, leading to delays and communication gaps.

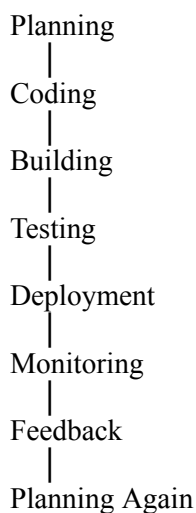
Traditional Software Development



Problems:

- Slow software delivery
- Manual deployments
- Communication gaps
- Frequent deployment failures
- High operational costs

DevOps Approach



Development and operations teams work together throughout the software lifecycle.



1.2 Goals of DevOps

The primary goals of DevOps are:

1. Faster software delivery.
 2. Improved collaboration between teams.
 3. Automation of repetitive tasks.
 4. Continuous Integration (CI).
 5. Continuous Delivery (CD).
 6. Better software quality.
 7. Faster issue resolution.
 8. Continuous monitoring and feedback.
 9. Reduced deployment failures.
 10. Improved customer satisfaction.
-

1.3 Benefits of DevOps

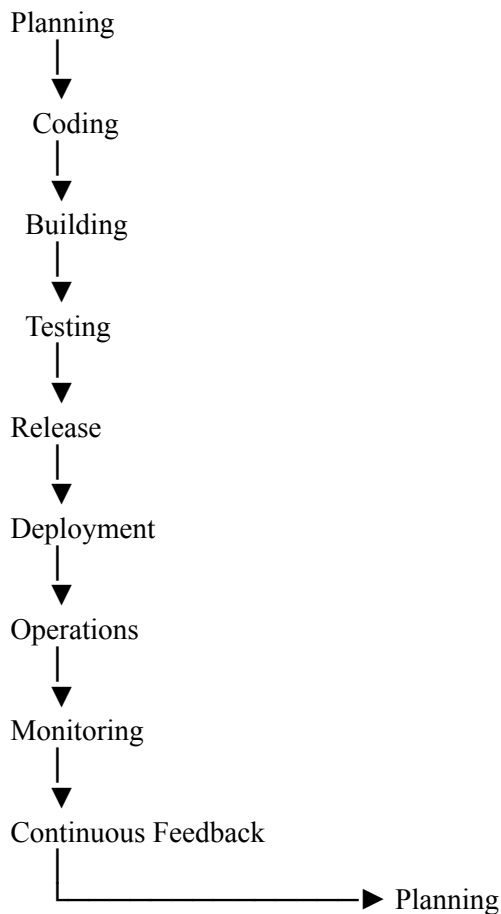
Benefit	Description
Faster Delivery	Software is released more frequently.
Improved Collaboration	Development and Operations work together.
Better Quality	Continuous testing reduces bugs.
Automation	Repetitive tasks are automated.
Reliability	Stable and consistent deployments.
Faster Recovery	Quick detection and resolution of failures.
Customer Satisfaction	Frequent feature updates improve user experience.
Cost Reduction	Automation reduces operational costs.

1.4 DevOps Lifecycle



The DevOps Lifecycle is a continuous process that automates software development from planning to monitoring.

Phases of DevOps Lifecycle



1. Planning: Requirements are gathered, tasks are prioritized, and project goals are defined.

Tools: Jira, Trello, Azure Boards

2. Coding : Developers write source code using programming languages and version control systems.

Tools: Git, GitHub, Visual Studio Code

3. Building : Source code is compiled into executable software.

Tools: Maven, Gradle, Ant

4. Testing : Automated tests verify software quality.

Tools: Selenium, JUnit, TestNG

5. Release : The software is prepared for deployment.

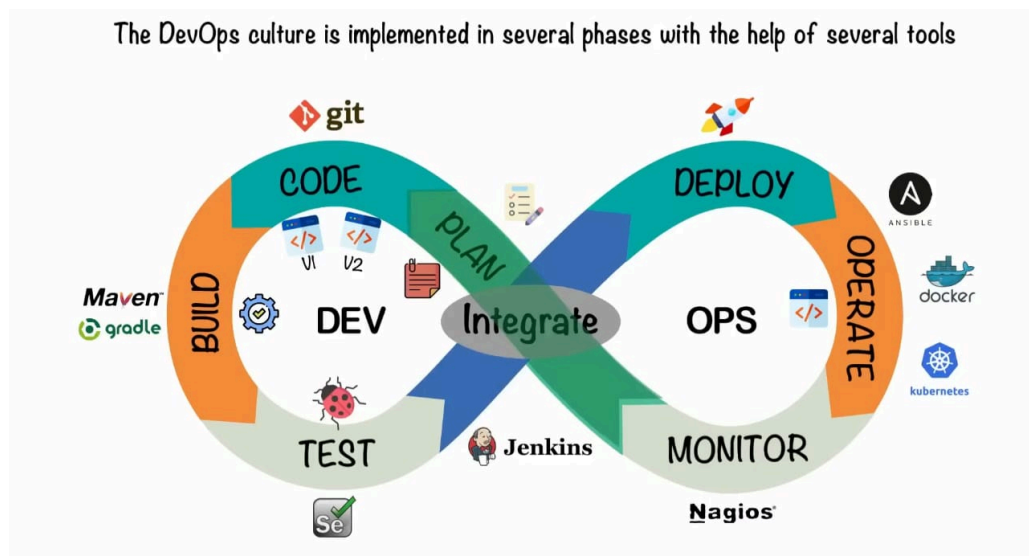
6. Deployment : Applications are deployed automatically.

Tools: Jenkins, Docker, Kubernetes

7. Operations : The application runs in production while ensuring reliability.

8. Monitoring : Performance and logs are continuously monitored.

Tools: Prometheus, Grafana, ELK Stack



1.5 Roles and Responsibilities of a DevOps Engineer

A DevOps Engineer bridges development and operations teams.

Responsibilities

- Automate software deployment.
- Develop CI/CD pipelines.
- Manage cloud infrastructure.
- Configure servers.
- Monitor applications.
- Troubleshoot production issues.
- Implement Infrastructure as Code.



- Manage ~~DOCKER and KUBERNETES~~.
- Ensure application security.
- Improve software performance.

Required Skills

- Linux
- Git
- Jenkins
- Docker
- Kubernetes
- AWS/Azure/GCP
- Python/Bash
- Networking
- Monitoring tools

1.6 DevOps Principles

Collaboration : Developers, testers, security, and operations teams work together to achieve common goals.

Automation : Tasks such as building, testing, deployment, and monitoring are automated to improve speed and consistency.

Continuous Improvement : Regular feedback and process optimization help improve software quality and team productivity.

Monitoring : Applications and infrastructure are continuously monitored to detect issues and improve performance.

1.7 Agile vs DevOps

Feature	Agile	DevOps
Focus	Software Development	Development + Operations
Objective	Fast software development	Fast software delivery and operations
Team	Developers and Testers	Developers, Testers, Operations, Security
Automation	Limited	Extensive
Deployment	Periodic	Continuous



Monitoring	Minimal	Continuous
Customer Feedback	End of Sprint	Continuous

1.8 Infrastructure as Code (IaC)

Definition

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure using code instead of manual configuration.

Traditional Infrastructure

- Manual server setup
- Time-consuming
- Error-prone

IaC Approach

Infrastructure is defined in configuration files, making deployments repeatable and consistent.

Benefits

- Automation
- Consistency
- Version control
- Scalability
- Faster deployment
- Reduced manual errors

Common IaC Tools

Tool	Purpose
Terraform	Cloud infrastructure provisioning
AWS CloudFormation	AWS infrastructure automation
Ansible	Configuration management
Puppet	Server configuration
Chef	Infrastructure automation



UNIT 2

Version Control Systems (Git)

2.1 Introduction to Version Control System (VCS)

A **Version Control System (VCS)** is software that records changes to source code or files over time, allowing multiple developers to collaborate efficiently and restore previous versions if needed.

Need for VCS

- Track changes
- Prevent overwriting
- Enable team collaboration
- Maintain version history
- Roll back to earlier versions

2.2 Types of Version Control Systems

Local Version Control System (LVCS)

Stores versions on a local computer.

Advantages: Simple, offline use.

Disadvantages: No collaboration.

Centralized Version Control System (CVCS)

A single central server stores all project files.

Examples: SVN, CVS

Advantages: Centralized management.

Disadvantages: Single point of failure.



Distributed Version Control System (DVCS)

Every developer has a complete copy of the repository.

Examples: Git, Mercurial

Advantages: Offline work, faster operations, better collaboration.

2.3 Git Fundamentals

Git is a free, open-source distributed version control system created by **Linus Torvalds** in 2005. It tracks file changes, supports branching and merging, and enables collaborative software development.

Features of Git

- Open source
- Distributed architecture
- Fast performance
- Lightweight
- Supports branching and merging
- Complete version history

2.4 Git Repository Concepts

A **repository (repo)** is a storage location that contains project files and their complete version history.

Types

- **Local Repository:** Stored on the developer's machine.
- **Remote Repository:** Hosted on servers such as GitHub, GitLab, or Bitbucket.

Git Workflow

Working Directory ↓ **Staging Area** ↓ **Local Repository** ↓ **Remote Repository**

2.5 Basic Git Commands

Command	Description
<code>git init</code>	Create a new Git repository
<code>git clone</code>	Copy an existing repository
<code>git add</code>	Stage files for commit



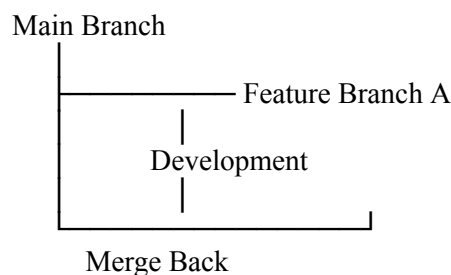
- `git commit` Save changes with a message
- `git push` Upload local commits to a remote repository
- `git pull` Download and merge changes from a remote repository
- `git branch` Create or list branches
- `git merge` Merge branches into one

2.6 Branching and Merging

Branching allows developers to work on new features independently without affecting the main codebase.

Merging combines changes from one branch into another after testing.

Branching Workflow



2.7 Repository Management

Repository management involves organizing and maintaining project repositories.

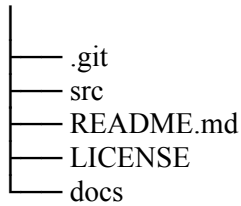
Best Practices

- Use meaningful commit messages.
- Create feature branches for new development.
- Merge only after testing.
- Protect the main branch.
- Review code before merging.
- Remove obsolete branches.
- Use `.gitignore` to exclude unnecessary files.
- Maintain clear project documentation.



Repository Structure

Project



The **.git** folder contains

- Commit history
- Branches
- Configuration
- Logs

Git Life Cycle

Create File



Modified



Staged



Committed



Pushed

5. Git Commands



1. git init

Creates a new Git repository.

Syntax

git init

Example

mkdir Project

cd Project

git init

Output

Initialized empty Git repository

2. git clone

Copies an existing repository.

Syntax

git clone URL

Example

git clone https://github.com/user/project.git

3. git status

Displays repository status.

Syntax

git status

Shows

- Modified files



- Untracked files
 - Staged files
-

4. git add

Adds files to staging area.

Single file

git add file.java

All files

git add .

5. git commit

Saves changes permanently.

Syntax

git commit -m "Added login module"

Example

Commit Message:

Fixed login bug

6. git log

Shows commit history.

git log

7. git push

Uploads local changes to remote repository.

Syntax



git push origin main

8. git pull

Downloads latest changes.

git pull origin main

9. git fetch

Downloads updates without merging.

git fetch

10. git branch

Lists branches.

git branch

Create branch

git branch feature-login

11. git checkout

Switch branch

git checkout feature-login

12. git switch

Modern command

git switch feature-login

13. git merge



Merge another branch.

git merge feature-login

14. git remote

Shows remote repositories.

git remote -v

15. git rm

Remove file.

git rm file.java

16. git mv

Rename file.

git mv old.java new.java

6. Branching Concepts

What is Branching?

A branch is an independent line of development.

It allows developers to work without affecting the main project.

Why Branch?

- Develop new features
- Fix bugs
- Test ideas
- Multiple developers



Branch Structure

Main

|

|---- Feature A

|

|---- Feature B

|

|---- Bug Fix

Default Branch

Usually

main

or

master

Branch Commands

Create

git branch feature1

Switch

git checkout feature1

Delete

git branch -d feature1

7. Merging Concepts



What is Merging?

Combining changes from one branch into another.

Example

Feature Branch

↓

Merge

↓

Main Branch

Types of Merge

Fast Forward Merge

Occurs when there are no conflicting commits.

Three-Way Merge

Occurs when both branches have changes.

Merge Conflict

Occurs when two developers modify the same line.

Example

Developer A

int marks = 80;

Developer B

int marks = 90;

Git cannot decide automatically.



Developer must resolve conflict manually.

Conflict Resolution

1. Open conflicting file
 2. Edit correct code
 3. Save file
 4. Stage changes
 5. Commit
-

8. Repository Management

Repository management refers to organizing, maintaining, and securing Git repositories effectively.

Best Practices

1. Meaningful Commit Messages

Good

Added Student Login Module

Bad

Update

2. Create README.md

Contains

- Project Description
 - Installation
 - Usage
 - Contributors
-



3. Use .gitignore

Ignore unnecessary files.

Example

*.class

*.exe

node_modules/

.idea/

4. Organize Branches

Example

main

develop

feature/login

feature/payment

bugfix/login

5. Regular Pull

Always pull latest changes before pushing.

git pull

6. Commit Frequently

Small commits are easier to understand and revert than one large commit.

7. Protect Main Branch

Only merge tested code into the **main** branch. Use code reviews and pull requests to maintain quality.



Common Git Workflow

Create Project



git init



Create/Edit Files



git status



git add .



git commit -m "Initial commit"



git branch feature-login



git switch feature-login



Develop Feature



git add .



git commit -m "Implemented login feature"



git switch main



git merge feature-login



git push origin main



Unit

Objectives

Design and implement a basic DevOps workflow using Git and Jenkins.

Tasks

Task 1: DevOps Planning

Prepare a report including:

- Introduction to DevOps
- Problems in the existing software development process
- Benefits of implementing DevOps
- DevOps lifecycle for the Student Management System

Task 2: Git Repository

Create a Git repository. Perform the following Git operations:

- git init
- git add
- git commit
- git branch
- git merge
- git push (GitHub)

Take screenshots of each command.

Task 3: Branch Management

Create three branches:

- main
- feature-login
- feature-dashboard

Merge all branches successfully. Prepare a branch diagram

Task 4: Jenkins

Install Jenkins. Create: One Freestyle Job | One Pipeline Job

Execute the pipeline after every Git commit.



Task 5: CI Pipeline

Design a CI pipeline showing

Developer

↓

Git

↓

Jenkins

↓

Build

↓

Testing

↓

Deployment

↓

Feedback

Deliverables

- Report (PDF)
- GitHub Repository Link
- Jenkins Screenshots
- Pipeline Diagram
- Viva Presentation

MINI PROJECT 2 : [Automated DevOps Workflow for an Online Shopping Website](#)

Scenario : An e-commerce company develops an online shopping website.



The company experiences.

- Frequent deployment failures
- Code conflicts
- Delayed software releases
- Manual testing
- No version control

As a DevOps Engineer, propose and implement a DevOps solution.

Objectives : Implement Version Control and Continuous Integration using Git and Jenkins.

Activities :

Phase 1 : Study the current software development process. Identify

- Problems
- Causes
- Solutions

Phase 2 : Create Git Repository . Maintain project source code. Perform

- Clone
- Add
- Commit
- Push
- Pull
- Merge

Phase 3 : Configure Jenkins . Create

- Build Job
- Test Job
- Pipeline Job

Phase 4 : Create Jenkinsfile . Include stages

- Checkout
- Build
- Test
- Deploy

Phase 5 : Explain Continuous Feedback . Prepare a flowchart showing how Jenkins informs developers about build failures.



- Expected Output
- Working CI Pipeline
- Git Repository
- Documentation
- Presentation

CASE STUDY 1 : Digital Banking Application

Background : A bank develops a mobile banking application.

Development team:

- 15 Developers
- Operations team:
- 5 Engineers

Problems:

- Software released every 4 months.
- Frequent production failures.
- Manual deployment.
- No automated testing.
- Code stored on individual systems.
- Difficult rollback after failures.

Questions

Part A : Identify at least five problems in the current process.

Part B : Explain how DevOps can solve these problems.

Part C : Design the DevOps lifecycle for the banking application.

Part D : Recommend

- Version Control System
- CI Tool
- Build Tool
- Monitoring Tool

Justify your choices.

Part E : Draw the complete CI/CD pipeline.

Part F : Explain the responsibilities of the DevOps Engineer in this project.



Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA

(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)

Approved by AICTE, Ministry of HRD, Government of India

Recognized by UGC under Section 2(f) of the UGC Act, 1956

Tel.: 91-0141-3500300 Fax: +91-0141-2759555

E-mail: info@skit.ac.in Web: www.skit.ac.in

Summary

- A **Version Control System (VCS)** tracks changes to files and supports collaboration.
- **Local VCS** stores versions on one computer, **Centralized VCS** uses a single server, and **Distributed VCS** (such as Git) gives every developer a complete repository copy.
- **Git** is the most widely used distributed version control system because it is fast, reliable, and supports offline work.
- A **repository** stores project files, commit history, branches, and configuration.
- Core Git commands include `git init`, `git clone`, `git add`, `git commit`, `git push`, `git pull`, `git branch`, and `git merge`.
- **Branching** allows independent development of features or bug fixes, while **merging** combines completed work back into the main branch.
- Good repository management includes meaningful commit messages, using `.gitignore`, maintaining documentation (`README.md`), organizing branches, pulling regularly, and protecting the main branch.



UNIT 3

Continuous Integration:

Jenkins & Build Automation:

Understanding CI/CD Pipelines: A Technical Briefing

Executive Summary

The transition from manual to automated software development processes is a fundamental shift in modern DevOps, primarily realized through CI/CD (Continuous Integration and Continuous Delivery/Deployment) pipelines. This document analyzes the core mechanics of CI/CD, the necessity of automation in preventing "Integration Hell," and the strategic methodologies used to deploy software at scale. By automating the build, test, and deployment phases, organizations achieve higher reliability, faster release cycles, and reduced human error. Key tools such as GitHub Actions and Jenkins, alongside deployment strategies like Blue-Green and Canary, provide the infrastructure for maintaining zero-downtime environments.

The Fundamentals of CI/CD

CI/CD stands for **Continuous Integration** and **Continuous Delivery** (or **Continuous Deployment**). It represents a sequence of automated steps—a pipeline—that code must pass through to move from a developer's local machine to a production environment.

The Software Development Life Cycle (SDLC)

In a modern development environment, applications are built incrementally through constant changes, bug fixes, and feature implementations. These changes typically follow a specific lifecycle:

1. **Development:** Writing the feature code.
2. **Commit and Push:** Sending code to a version control system (like Git). Developers work on specific "branches" before merging into a "main branch."
3. **Build Stage:** Creating "deployable artifacts." This involves compiling source code and including dependencies to create executable files, such as **.apk** files for Android.
4. **Testing:** Running automated checks to ensure the code works as intended.
5. **Release/Deployment:** Making the application available to end-users across different environments.

Deployment Environments



Organizations typically maintain three distinct environments to manage risk.

- **Dev (Development):** Used exclusively by developers.
- **Staging (Pre-production):** A replica of the production environment used for final demos, manager approvals, and end-to-end testing (security, performance, and smoke tests).
- **Prod (Production):** The live environment accessible to all end-users.

The Risks of Manual Processes

Before the adoption of CI/CD, the software delivery process was largely manual, leading to several critical inefficiencies:

- **Integration Hell:** When multiple developers work on a single application, merging their code manually often results in conflicts and errors that are time-consuming to resolve.
- **Human Error:** Manual testing is inconsistent and prone to oversight.
- **Infrequent Releases:** Because manual testing and deployment require "code freezes" and significant labor, organizations cannot release updates frequently.
- **Reliability Issues:** If a manual deployment fails on a Friday or weekend, it may result in prolonged downtime because no developers are available to manually restore a backup.

Detailed Component Analysis

Continuous Integration (CI)

Continuous Integration focuses on the early stages of the pipeline. It automates the **Build** and **Test** phases. Every time a developer pushes code to the repository, the CI system automatically:

- Builds the application to ensure there are no compilation errors.
- Runs **Unit Tests** (basic checks) and **Integration Tests** (checking how components work together).
- Performs **Regression Testing** to ensure new changes haven't broken existing functionality.

If any test fails, the developer is notified immediately to fix the code before it is integrated into the main branch.

Continuous Delivery vs. Continuous Deployment (CD)

While "CD" is used as a generic term, there is a technical distinction based on the level of automation at the final production stage:

Feature	Continuous Delivery	Continuous Deployment
Automation Level	Automated through Staging.	Fully automated through Production.



Manual Step	Requires manual approval (e.g., from a manager or CTO) before moving to Production.	No manual approval; code goes live immediately after passing tests.
Use Case	Ideal for sensitive data (Banking/FinTech).	Ideal for high-velocity services (Netflix/Hotstar).

Industry Tools and Implementation

CI/CD pipelines are implemented using automation scripts, often written in **YAML** (Yet Another Markup Language). These scripts provide instructions on how to build and test the code.

Popular CI/CD Tools

Tool	Characteristics	Recommended Use Case
GitHub Actions	Native to GitHub; released in 2019; highly intuitive and easy to set up.	Organizations already using GitHub for version control.
Jenkins	Released in 2011; a "job orchestrator"; highly customizable but harder to maintain.	Established organizations with complex, legacy customization needs.
Other Tools	CircleCI, TravisCI, GitLab CI/CD, Bamboo.	Dependent on specific organizational infrastructure.

Deployment Strategies for Zero Downtime

To avoid application crashes or buggy releases, teams utilize specific strategies to manage how new versions of software are introduced to users.

- **Blue-Green Deployment:** Two identical environments are maintained. The "Blue" environment runs the current version, and "Green" runs the new version. Once the Green version is verified, all traffic is redirected to it. If an error occurs, traffic is instantly switched back to Blue.
- **Canary Deployment:** The new version is released to a small percentage of users (e.g., 5-10%) for a few hours. If no issues are detected, the new version is progressively rolled out to the rest of the user



base.

- **Rolling Deployment:** The new version is deployed instance-by-instance or server-by-server. This allows the organization to test the new version on specific instances while the old version remains live on others.

Conclusion

CI/CD is a core pillar of DevOps that transforms software development from a slow, error-prone manual process into a fast, reliable, and automated system. By utilizing tools like GitHub Actions and advanced deployment strategies, development teams can ensure that their applications remain scalable and available to users with minimal risk of downtime.

1. What is Continuous Integration (CI)?

Answer:

Continuous Integration (CI) is a DevOps practice in which developers frequently merge their code changes into a shared repository. Every time code is committed, an automated process builds the application and runs tests to detect errors early.

Instead of waiting until the end of development to integrate code, CI encourages developers to integrate their work several times a day.

Key Features of CI

- Frequent code integration
- Automated builds
- Automated testing
- Early bug detection
- Immediate feedback to developers

Example

Suppose five developers are working on an online shopping website. Each developer commits code to GitHub daily. Jenkins automatically builds the project and runs tests after every commit. If an error is detected, Jenkins immediately notifies the developer.

Benefits

- Detects bugs early
- Improves software quality
- Reduces integration conflicts
- Speeds up software delivery
- Saves development time



Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel.: 91-0141-3500300 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

How Does CI/CD Work with Git?

- Every time you push code to your Git repository:
- The CI/CD service (like GitHub Actions or GitLab CI) detects the change
- It runs tests, builds your project, and can deploy automatically
- If something fails, you get notified right away

Popular CI/CD Services

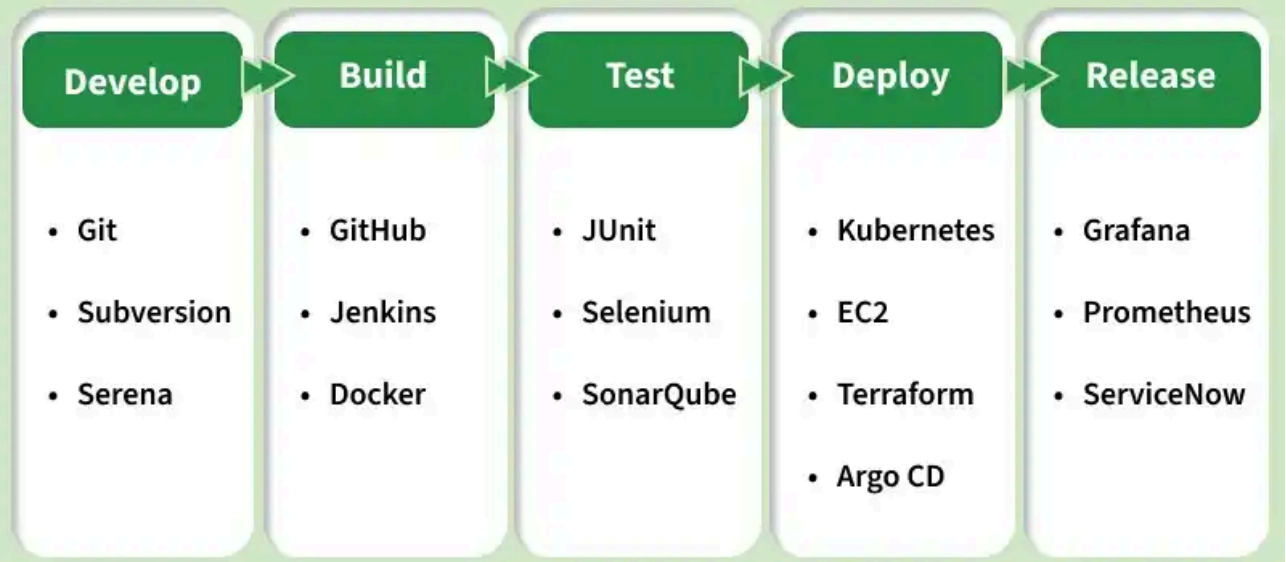
- **GitHub Actions:** Built into GitHub, uses YAML files in `.github/workflows/`
- **GitLab CI/CD:** Built into GitLab, uses `.gitlab-ci.yml`
- **CircleCI:** Works with GitHub/GitLab, easy setup for many languages
- **Travis CI:** Popular for open-source, uses `.travis.yml`
- **Azure Pipelines:** Works with Azure DevOps and GitHub, supports many platforms

Key CI/CD Concepts

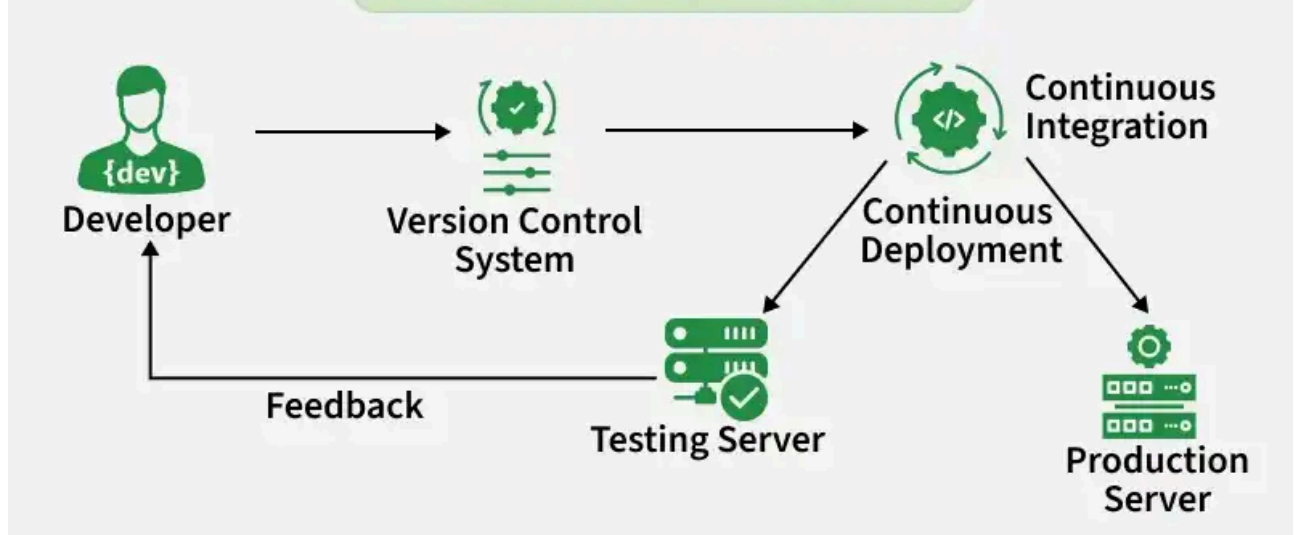
Here are some important terms:

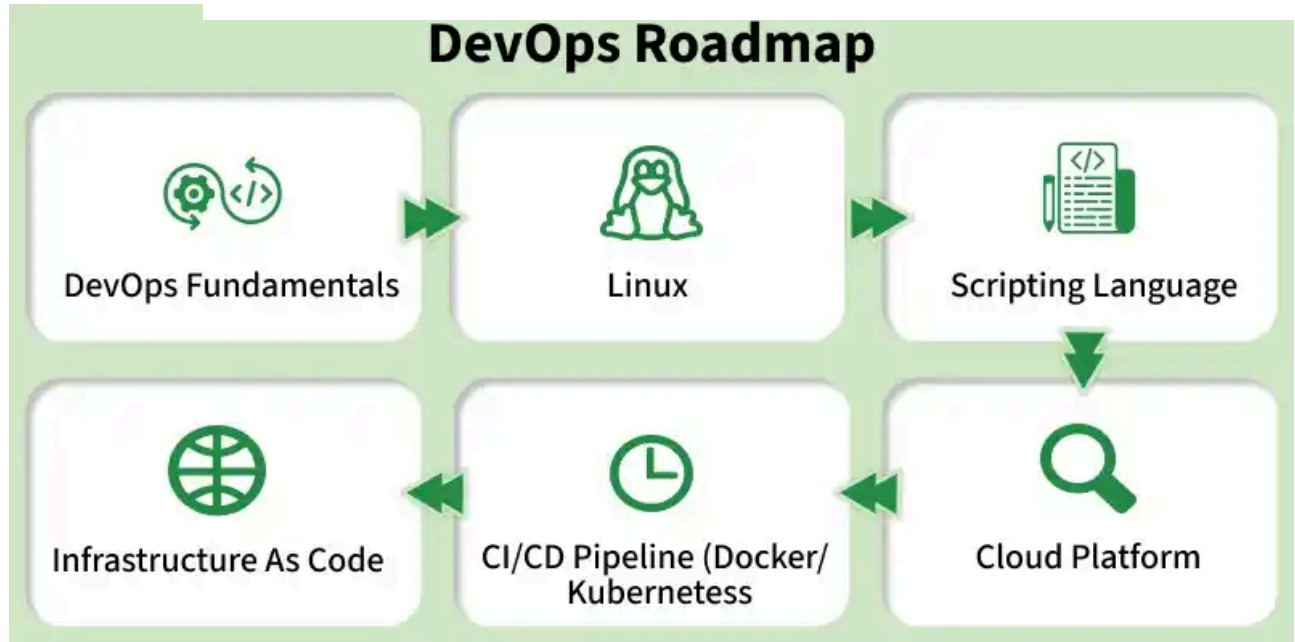
- **Workflow:** A series of jobs that run together
- **Job:** A group of steps that run together
- **Step:** A single task, like checking out code or running tests
- **Runner:** The computer/server that runs your jobs
- **Trigger:** Decides when your workflow runs
- **Environment Variables:** Settings for your workflow
- **Secrets:** Passwords or API keys

Common DevOps Tools



How DevOps Works





2. Why is Jenkins used in a CI pipeline?

Answer:

Jenkins is an open-source automation server that automates software development tasks such as building, testing, and deploying applications.

It acts as the central automation tool in a CI pipeline.

Why Jenkins is Used

- Automatically detects code changes
- Builds the application
- Runs automated tests
- Generates reports
- Deploys applications
- Sends notifications if builds fail
- Integrates with Git, GitHub, Maven, Docker, Kubernetes, and cloud platforms

Example

When a developer pushes code to GitHub, Jenkins automatically:



1. Downloads the latest code.
2. Compiles the application.
3. Executes test cases.
4. Creates deployment files.
5. Notifies the team about the build status.

Advantages

- Free and open source
 - Thousands of plugins
 - Platform independent
 - Easy integration with DevOps tools
 - Supports CI/CD pipelines
-

3. What is the role of a GitHub webhook in Jenkins?

Answer:

A **GitHub webhook** is a mechanism that automatically informs Jenkins whenever a change occurs in a GitHub repository.

Instead of Jenkins repeatedly checking GitHub for updates, GitHub sends an HTTP request (webhook) to Jenkins immediately after events such as a code push.

Workflow

1. Developer commits code.
2. Developer pushes code to GitHub.
3. GitHub sends a webhook notification to Jenkins.
4. Jenkins starts the CI pipeline automatically.

Benefits

- Real-time automation
- Faster builds
- Eliminates unnecessary polling
- Reduces server load

Example

Whenever a new commit is pushed to the **main** branch, GitHub instantly triggers Jenkins to start a new build.



4. What are the major stages of a Jenkins pipeline?

Answer:

A Jenkins pipeline automates the software development workflow through several stages.

Major Stages

Stage	Description
Checkout	Downloads the latest source code from GitHub.
Build	Compiles the application using tools like Maven or Gradle.
Test	Runs automated unit and integration tests.
Package	Creates deployable artifacts such as JAR or WAR files.
Deploy	Deploys the application to a test or production server.
Post	Performs actions such as sending notifications or cleaning up resources.

Example Pipeline

```
Checkout
↓
Build
↓
Test
↓
Package
↓
Deploy
↓
Post
```

5. What is the difference between `mvn compile`, `mvn test`, and `mvn package`?



Answer:

These are common Maven lifecycle commands.

Command	Purpose	Output
<code>mvn compile</code>	Compiles the project's source code.	Compiled <code>.class</code> files
<code>mvn test</code>	Compiles the code and executes unit tests.	Test reports
<code>mvn package</code>	Compiles, tests, and packages the application into a deployable file.	JAR/WAR file

Example

Compile

`mvn compile`

Only checks whether the source code compiles successfully.

Test

`mvn test`

Runs all unit tests after compilation.

Package

`mvn package`

Produces a deployable JAR or WAR file after successful compilation and testing.

6. Why is automated testing important in CI?

Answer:

Automated testing executes predefined test cases automatically whenever code changes are made.



Importance

- Detects bugs early
- Improves software quality
- Reduces manual testing effort
- Speeds up development
- Prevents faulty code from reaching production

Types of Automated Testing

- Unit Testing
- Integration Testing
- Functional Testing
- Regression Testing

Example

After every code commit, Jenkins automatically runs JUnit tests. If any test fails, the build is marked as failed.

7. What happens if a Jenkins build fails?

Answer:

If a Jenkins build fails, the CI pipeline stops, and Jenkins reports the failure to the development team.

Actions Performed

- Marks the build as **Failed**.
- Displays detailed error logs.
- Sends notifications (email, Slack, Microsoft Teams, etc.).
- Prevents deployment of faulty software.
- Allows developers to fix issues before continuing.

Example

If a compilation error occurs because of a syntax mistake, Jenkins stops the pipeline and reports the error so the developer can correct it.

8. What is the purpose of the **post** section in a



Jenkinsfile?

Answer:

The **post** section defines actions that should run after the pipeline or a stage completes, regardless of whether it succeeds or fails.

Common Blocks

```
post {  
    success {  
        echo 'Build Successful'  
    }  
  
    failure {  
        echo 'Build Failed'  
    }  
  
    always {  
        echo 'Pipeline Completed'  
    }  
}
```

Benefits

- Sends notifications
- Cleans temporary files
- Archives reports
- Publishes test results
- Performs cleanup activities

Example

After every pipeline execution, Jenkins sends an email to developers with the build result.

9. How does Jenkins improve software quality?

Answer:

Jenkins improves software quality by automating repetitive development tasks and ensuring that code changes are validated continuously.

Ways Jenkins Improves Quality



- Automates builds
- Runs automated tests
- Detects bugs early
- Enforces consistent build processes
- Provides quick feedback
- Integrates with code quality tools such as SonarQube
- Reduces human errors

Example

A developer accidentally introduces a bug. Jenkins immediately runs tests after the code is committed, detects the issue, and alerts the developer before deployment.

Benefits

- Higher code quality
- Faster issue resolution
- Reliable software releases
- Increased team productivity

10. Differentiate between Continuous Integration (CI) and Continuous Deployment (CD)

Answer:

Feature	Continuous Integration (CI)	Continuous Deployment (CD)
Definition	Frequently integrates code changes into a shared repository with automated builds and tests.	Automatically deploys every successful build to production without manual approval.
Primary Goal	Detect integration issues early and ensure code quality.	Deliver new features and fixes to users as quickly as possible.
Trigger	Code commit or pull request.	Successful completion of the CI pipeline.
Testing	Automated builds and tests are mandatory.	Deployment occurs only after all CI tests pass.



Deployment	May require manual approval for release.	Deployment is fully automated.
Human Intervention	Usually required before production deployment.	No manual intervention for production deployment.
Outcome	Produces a tested and deployable build.	Delivers the tested build directly to end users.

CI Workflow

Developer
↓
Git Repository
↓
Jenkins Build
↓
Automated Testing
↓
Ready for Deployment

CD Workflow

Developer
↓
Git Repository
↓
Jenkins Build
↓
Automated Testing
↓
Automatic Deployment
↓
Production Environment

DevOps: Build Automation, Automated Testing in CI, and Continuous Feedback

These three concepts are closely connected in a **Continuous Integration (CI)** pipeline:

Developer commits code → Build automatically → Test automatically → Generate feedback → Fix



issues → Repeat

1. Build Automation Concepts

What is Build Automation?

Build automation is the process of automatically converting source code into a software product that can be tested or deployed.

Instead of a developer manually performing tasks such as:

- Compiling source code
- Downloading dependencies
- Running tests
- Creating JAR/WAR files
- Packaging the application
- Generating build reports

a **build automation tool** performs these tasks automatically.

Simple Example

Suppose a Java developer writes:

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Normally, a developer might manually compile it:

```
javac Calculator.java
```

In a DevOps environment, a build tool such as **Maven** can automate the process.

```
mvn clean package
```

Maven can:

1. Clean previous build files.
2. Download required dependencies.
3. Compile the source code.
4. Run tests.
5. Package the application.



6. Generate the final artifact.

Why Do We Need Build Automation?

Manual builds can create several problems.

1. Human errors

A developer may forget to compile a particular file or include a dependency.

2. Time-consuming

Manually building a large application can take significant time.

3. Inconsistent builds

Different developers may use different commands or configurations.

4. Difficult to repeat

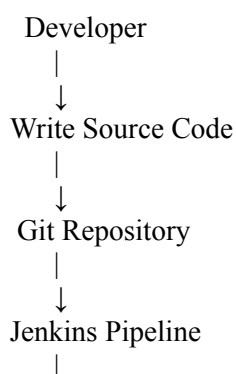
A build should produce the same result whenever the same source code and configuration are used.

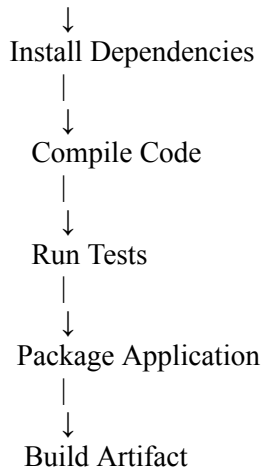
5. Poor integration with CI/CD

Continuous Integration requires builds to happen automatically whenever code changes.

Build Automation Process

A typical automated build looks like this:





Common Build Automation Tools

Some commonly used tools are:

Tool	Commonly Used For
Maven	Java
Gradle	Java/Kotlin
npm	JavaScript/Node.js
MSBuild	.NET
Ant	Java
Make	C/C++ and Unix-based projects

Maven Example

A Java project commonly contains a file called:

pom.xml

This file contains project configuration and dependencies.

For example:



```
<dependency>  
  <groupId>junit</groupId>  
  <artifactId>junit</artifactId>  
  <version>5.10.0</version>  
</dependency>
```

Jenkins can execute:

bat 'mvn clean package'

on Windows.

On Linux:

sh 'mvn clean package'

Meaning

mvn

↓

Maven

clean

↓

Remove previous build output

package

↓

Compile + test + package application

2. Automated Testing in CI Pipeline

What is Automated Testing?

Automated testing means using software tools to automatically execute test cases and verify whether the application behaves as expected.

Instead of manually testing every feature after every code change, automated tests can be executed automatically.

Why is Automated Testing Important in CI?



Imagine 20 developers are working on the same application.

Every developer commits code several times a day.

If testers manually test the entire application after every commit, it becomes extremely difficult.

CI solves this problem:

Developer Commit



CI Pipeline



Build



Automated Tests



Pass / Fail

If the tests fail, developers receive immediate feedback.

Types of Automated Testing

1. Unit Testing

Tests individual methods or functions.

Example:

```
@Test
void testAddition() {
    Calculator c = new Calculator();

    assertEquals(10, c.add(5, 5));
}
```

This checks whether:

$5 + 5 = 10$

2. Integration Testing

Tests whether multiple components work together correctly.

For example:



Java Application



Database



Payment Service

Integration testing checks whether these components communicate correctly.

3. System Testing

Tests the complete application as a whole.

For example:

Login



Dashboard



Product Selection



Payment



Order Confirmation

4. Regression Testing

Regression testing ensures that **new code has not broken existing functionality**.

For example:

An existing banking application has:

Login

Balance Check

Fund Transfer

Statement

A developer adds a new feature:

UPI Payment

Regression tests check whether the existing features still work.



Automated Testing in Jenkins

A Jenkins pipeline may contain:

```
pipeline {  
    agent any  
  
    stages {  
  
        stage('Build') {  
            steps {  
                bat 'mvn clean package'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                bat 'mvn test'  
            }  
        }  
    }  
}
```

Here:

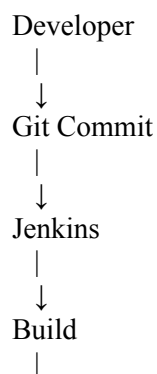
```
bat 'mvn test'
```

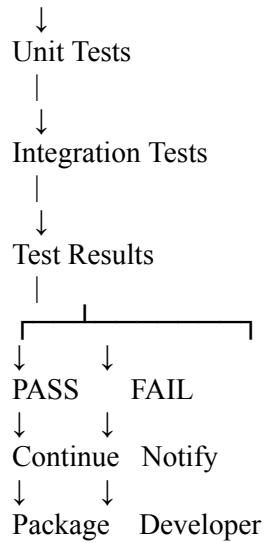
automatically executes the project's tests.

For Linux:

```
sh 'mvn test'
```

CI Testing Flow





What Happens When a Test Fails?

Suppose a developer changes:

return a - b;

instead of:

return a + b;

The automated test expects:

$5 + 5 = 10$

but receives:

$5 - 5 = 0$

Therefore:

Test Failed



Jenkins Build Failed



Developer Notified



Developer Fixes Code



Commit Again





Pipeline Runs Again

This is one of the major benefits of CI.

3. Continuous Feedback in CI Process

What is Continuous Feedback?

Continuous feedback means providing developers with immediate and continuous information about the status and quality of their code.

The feedback can tell developers:

- Whether the build succeeded
 - Whether tests passed
 - Which tests failed
 - Whether code quality is acceptable
 - Whether security checks passed
 - Whether deployment was successful
-

Why is Continuous Feedback Important?

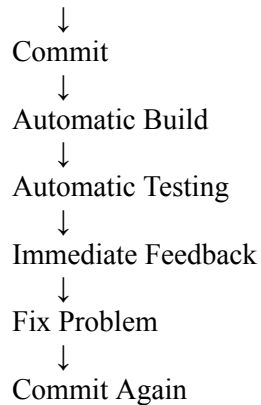
Without continuous feedback:

Developer writes code
↓
Code submitted
↓
Wait several days
↓
Testing
↓
Problem discovered
↓
Developer fixes problem

This creates delays.

With CI:

Developer writes code



The problem is identified much earlier.

Types of CI Feedback

1. Build Feedback

Jenkins reports:

BUILD SUCCESS

or:

BUILD FAILED

2. Test Feedback

For example:

Tests run: 100

Passed: 97

Failed: 3

The developer immediately knows that three tests failed.

3. Code Quality Feedback

Tools such as SonarQube can identify:



- Bugs
- Code smells
- Duplicated code
- Security vulnerabilities
- Maintainability issues

4. Notification Feedback

Jenkins can notify developers through various communication channels when a pipeline succeeds or fails.

For example:

Build #25

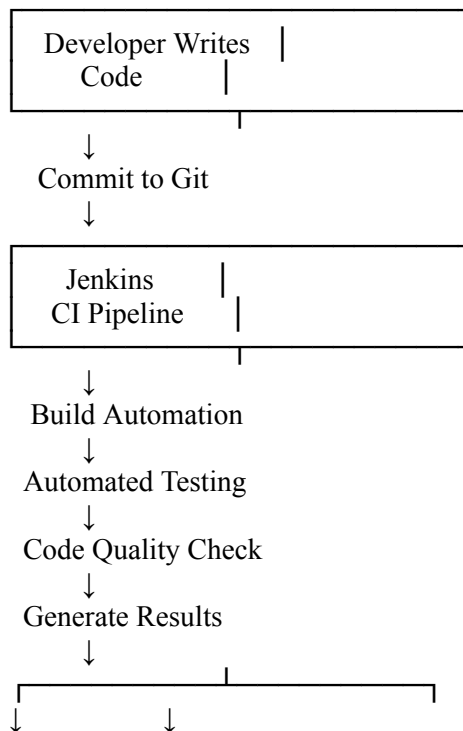
Status: FAILED

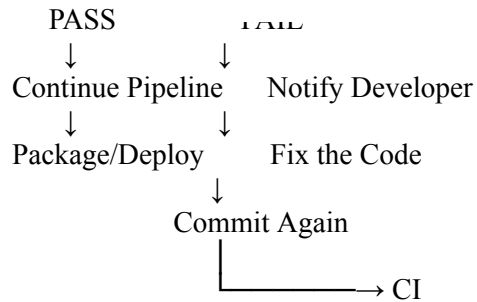
Reason:

3 automated tests failed.

Continuous Feedback Loop

The entire CI process can be represented as:





Build Automation vs Automated Testing vs Continuous Feedback

Concept	Meaning	Example
Build Automation	Automatically builds the application	<code>mvn clean package</code>
Automated Testing	Automatically executes test cases	<code>mvn test</code>
Continuous Feedback	Automatically informs developers about results	Jenkins build/test report

Easy way to remember

Build → Test → Feedback

BUILD
↓
Create software
↓
TEST
↓
Check software
↓
FEEDBACK
↓
Tell developer the result

Complete Jenkins CI Example

```
pipeline {
```



agent any

stages {

```
stage('Checkout') {  
  steps {  
    git 'https://github.com/example/student-project.git'  
  }  
}
```

```
stage('Build') {  
  steps {  
    bat 'mvn clean package'  
  }  
}
```

```
stage('Test') {  
  steps {  
    bat 'mvn test'  
  }  
}
```

```
stage('Feedback') {  
  steps {  
    echo 'Build and test process completed.'  
  }  
}  
}
```

What happens?

Stage 1 – Checkout

Jenkins gets the latest source code from Git.

Stage 2 – Build

mvn clean package

builds the Java application.

Stage 3 – Test

mvn test

runs automated tests.

Stage 4 – Feedback



Jenkins provides the build and test status.

Real-Life Example: Online Shopping Website

Suppose an e-commerce team adds a **new discount feature**.

Step 1 – Developer writes code

Discount.java

Step 2 – Developer commits code

git commit
git push

Step 3 – Jenkins detects the change

Git → Jenkins

Step 4 – Build automation

mvn clean package

Step 5 – Automated tests

Jenkins checks:

Login Test	✓
Product Test	✓
Cart Test	✓
Payment Test	✓
Discount Test	✗

Step 6 – Continuous feedback

Jenkins reports:

BUILD FAILED

Discount Test Failed
Expected: ₹900
Actual: ₹1000

The developer immediately understands the problem and fixes the discount calculation.



Key Exam Points

Build Automation

Build automation is the process of automatically compiling, testing, packaging, and generating software artifacts using build tools.

Automated Testing in CI

Automated testing in CI automatically executes predefined test cases whenever new code is integrated, helping detect defects at an early stage.

Continuous Feedback

Continuous feedback provides developers with immediate information about build, test, quality, and deployment results so that problems can be corrected quickly.

Most Important CI Cycle

```
CODE
↓
COMMIT
↓
BUILD
↓
TEST
↓
FEEDBACK
↓
FIX
↓
COMMIT AGAIN
```

Build automation, automated testing, and continuous feedback are the core pillars of a modern Continuous Integration (CI) pipeline. Together, they transform manual, error-prone software development into a fast, repeatable, and high-quality deployment machine.

Here is a detailed breakdown of each concept.

1. Build Automation Concepts

Build automation is the practice of scripting and automating the process of preparing source code for production. Instead of a developer manually compiling code and handling



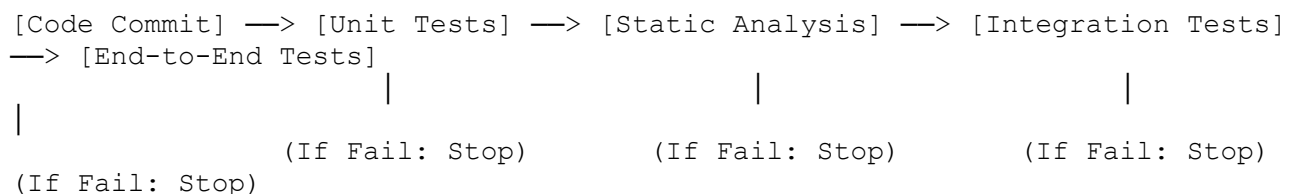
files, specialized build tools do it automatically upon every code commit.

Key Steps in Build Automation

- **Compilation:** Translating human-readable source code into machine-readable binaries or bytecode (e.g., converting Java code into `.class` files).
 - **Dependency Management:** Automatically identifying, downloading, and caching external libraries required by the application using package managers.
 - **Code Linting:** Analyzing source code to enforce strict formatting standards and detect syntax anomalies before compilation.
 - **Packaging:** Consolidating compiled code, media assets, and configuration files into an executable format like a `.jar` archive, a `.msi` installer, or a containerized Docker image.
 - **Artifact Storage:** Pushing the final packaged file (the "artifact") to a secure repository manager like JFrog Artifactory or AWS CodeArtifact for later deployment stages.
 - **Popular Build Automation Tools**
 - **Java/JVM Ecosystem:** [Apache Maven](#) and [Gradle](#).
 - **JavaScript/Node.js Ecosystem:** [NPM](#), [Yarn](#), and [Webpack](#).
 - **Python Ecosystem:** [Pipenv](#) and [Poetry](#).
-

2. Automated Testing in a CI Pipeline

Automated testing is the execution of a predefined suite of tests without any human intervention. In a CI pipeline, testing must follow a strict "fail-fast" order. Tests are structured sequentially—from the fastest to the slowest—to catch bugs as early as possible. [1, 2, 3, 4]



Test Hierarchy in CI

1. **Unit Testing:** Tests individual functions or modules in complete isolation. These run in seconds using tools like JUnit or PyTest.
2. **Static Code Analysis:** Scans source code for security vulnerabilities, memory leaks, or poor architecture without actually executing the program, using tools like SonarQube.
3. **Integration Testing:** Validates that independent modules, microservices, or



databases interact correctly.

4. **End-to-End (E2E) Testing:** Simulates entire user workflows in a browser or application window using tools like [Selenium](#) or [Playwright](#). [1, 2, 3, 4, 5]

Execution Best Practices

- **Parallel Testing:** Running independent tests simultaneously on different servers to slash total execution times.
- **Immutability:** Spinning up fresh, disposable test environments using Docker containers to guarantee a consistent testing state every single time. [1, 2, 3, 4, 5]

3. Continuous Feedback in the CI Process

Continuous feedback is the loop that immediately alerts the development team of the pipeline's status. If a build breaks or a test fails, developers must know within minutes so they can fix the issue while the code is still fresh in their minds. [1, 2, 3]

How Continuous Feedback Works

- **Real-Time Communication:** Integrations broadcast pipeline health metrics directly to communication hubs like Slack or Microsoft Teams.
- **Pull Request (PR) Gates:** Orchestrators like [GitHub](#) block developers from merging code if the automated CI pipeline fails.
- **Detailed Logs:** Code execution systems capture complete stack traces, console outputs, and screen video recordings of failing E2E tests.
- **Actionable Reports:** Specialized reporting tools parse data to display test coverage percentages, architectural regression warnings, and security threat scores. [1, 2, 3]

Value to the Business

- **Saves Money:** Finding a bug during the commit stage costs exponentially less to fix than finding it after it reaches production.
- **Improves Collaboration:** Clear, objective telemetry replaces finger-pointing with direct, undeniable data points. [1, 2, 3, 4, 5]



Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel.: 91-0141-3500300 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

What is Jenkins?

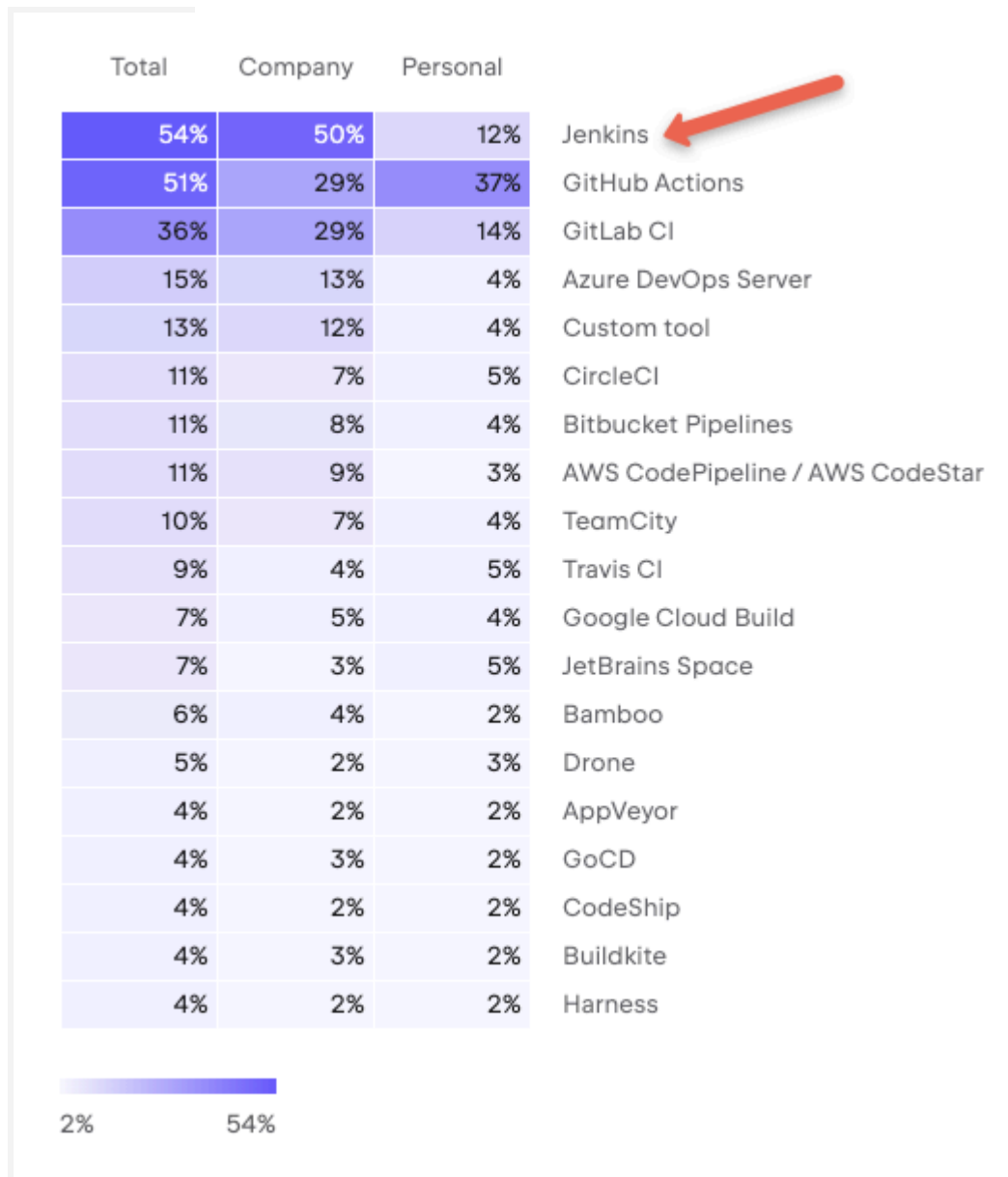
Jenkins is an easy-to-use open-source java-based **CI/CD** tool. It has been around for some time, and several organizations use it for their CI/CD needs.

Important Note: It is essential to have an understanding of Continuous integration & continuous delivery to understand Jenkins better.

Jenkins has huge community support and an ocean of plugins that can integrate with many open-source and enterprise tools to make your life so easy.

Despite the growing popularity of CI/CD tools like GitLab CI and GitHub Actions, which are increasingly being adopted by organizations for their CI/CD needs, Jenkins remains widely used in many organizations.

As per the **Developer Ecosystem Report**, 54% of the developers use Jenkins for CI/CD.



Jenkins is commonly used for the following.

1. Continuous Integration for application and infrastructure code.
2. Continuous delivery pipelines to deploy the application to different environments using **Jenkins pipeline as code**.
3. Infrastructure component deployment and management using **IaC** Tools..
4. Run batch operations using Jenkins jobs.



**Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA**

(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)

Approved by AICTE, Ministry of HRD, Government of India

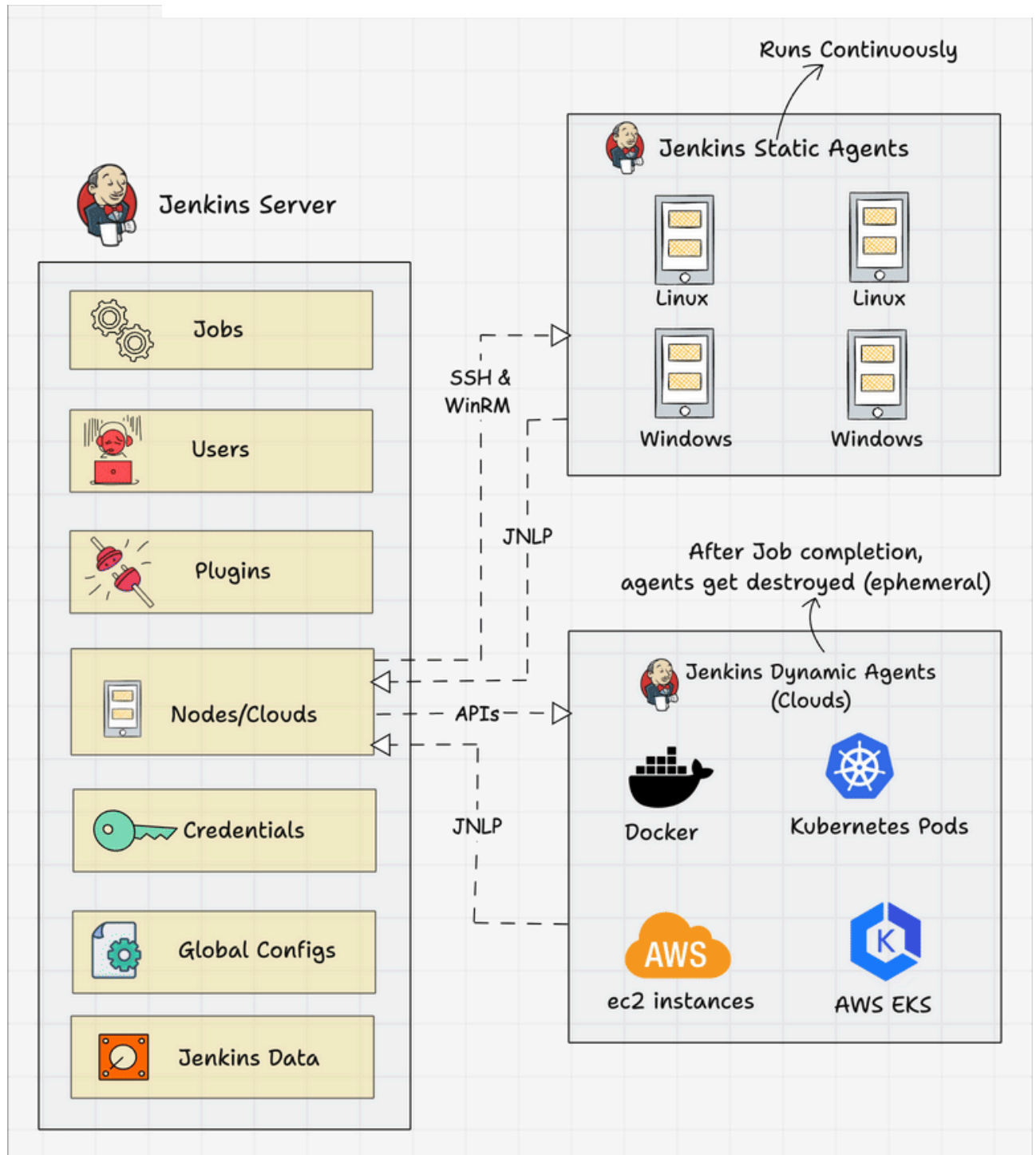
Recognized by UGC under Section 2(f) of the UGC Act, 1956

Tel.: 91-0141-3500300 Fax: +91-0141-2759555

E-mail: info@skit.ac.in Web: www.skit.ac.in

5. Run ad-hoc operations like backups, cleanups, remote script execution, event triggers, etc.
6. Jenkins Architecture

The following diagram shows the overall architecture of Jenkins and the connectivity workflow..



Following are the key components in Jenkins

1. Jenkins Master Node



**Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA**

(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)

Approved by AICTE, Ministry of HRD, Government of India

Recognized by UGC under Section 2(f) of the UGC Act, 1956

Tel.: 91-0141-3500300 Fax: +91-0141-2759555

E-mail: info@skit.ac.in Web: www.skit.ac.in

2. Jenkins Agent Nodes/Clouds
3. Jenkins Web Interface

Let's look at each component in detail.

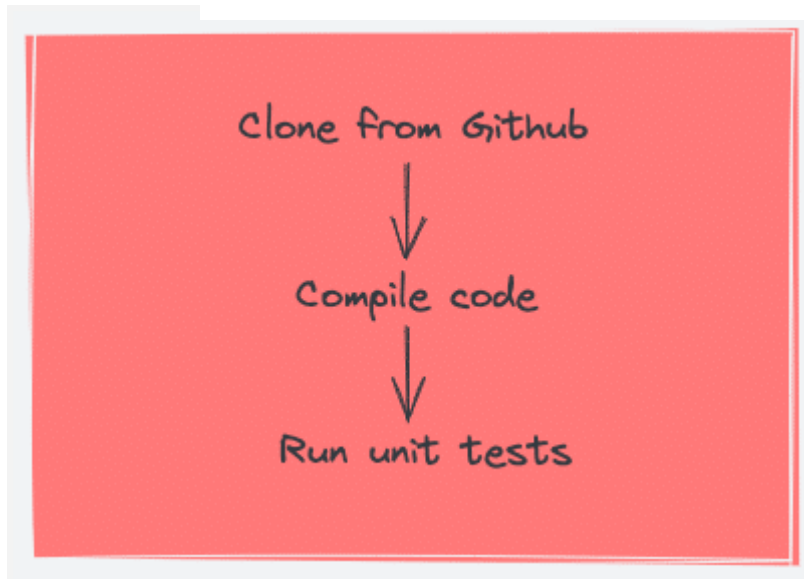
Jenkins Server (Formerly Master)

Jenkins's server or master node holds all key configurations. Jenkins master server is like a control server that orchestrates all the workflow defined in the pipelines. For example, scheduling a job, monitoring the jobs, etc.

Let's have a look at the key Jenkins master components.

Jenkins Jobs

A job is a collection of steps that you can use to build your source code, test your code, run a shell script, run an Ansible role in a remote host or execute a terraform play, etc. We normally call it a **Jenkins pipeline**.



If you translate the above steps to a Jenkins pipeline job, it looks like the following.

```
stage('Code Checkout') { ←
    steps {
        checkout([
            $class: 'GitSCM',
            branches: [[name: '*/master']],
            userRemoteConfigs: [[url: 'https://github.com/spring-
projects/spring-petclinic.git']]
        ])
    }
}

stage('Code Build') { ←
    steps {
        sh 'mvn install -Dmaven.test.skip=true'
    }
}
```

There are multiple job types available to support your workflow for continuous integration & continuous delivery.



Jenkins Plugins

Plugins are official and community-developed modules that you can install on your Jenkins server. It helps you with more functionalities that are not natively available in Jenkins.

For example, if you want to upload a file to s3 bucket from Jenkins, you can install an AWS Jenkins plugin and use the abstracted plugin functionalities to upload the file rather than writing your own logic in AWS CLI. The plugin takes care of error and exception handling.

Here is an example, of s3 file upload functionality provided by the **AWS Steps plugin**

```
s3Upload(  
  file:'file.txt',  
  bucket:'my-bucket',  
  path:'path/to/target/file.txt'  
)  
  
s3Upload(  
  file:'someFolder',  
  bucket:'my-bucket',  
  path:'path/to/targetFolder/'  
)
```

You can install/upgrade all the available plugins from the Jenkins dashboard itself.



Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel.: 91-0141-3500300 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

For corporate network, you will have to setup a proxy details to connect to the plugin repository.

You can also download the plugin file and install it by copying it to the plugins directory under `/var/lib/jenkins` folder.

You can also develop your custom plugins. Check out all plugins from the [Jenkins Plugin Index](#)

Jenkins Global Security

Jenkins has the following type of primary authentication methods.

1. **Jenkins's own user database:-** Set of users maintained by Jenkins's own database. When we say database, its all flat config files (XML files).
2. **LDAP Integration:-** Jenkins authentication using corporate LDAP configuration.
3. **SAML Single Sign On(SSO):** Support single signon using providers like Okta, AzureAD, Auth0 etc..

With Jenkins matrix-based security you can further assign roles to users on what permission they will have on Jenkins.

Jenkins Credentials



When you set up Jenkins pipelines, there are scenarios where it needs to connect to a cloud account, a server, a database, or an API endpoint using secrets.

In Jenkins, you can save different types of secrets as a credential.

1. Secret text
2. Username & password
3. SSH keys

All credentials are encrypted (AES) by Jenkins. The secrets are stored in `$JENKINS_HOME/secrets/` directory. It is very important to secure this directory and exclude it from Jenkins backups.

Note: Best practice is to use external secrets management solutions like **vault** instead of Jenkins native credentials.

Jenkins Nodes/Clouds

You can configure multiple agent nodes (Linux/Windows) or clouds (**docker**, kubernetes) for executing Jenkins jobs. We will learn more about it in the agent section.

Jenkins Global Settings (Configure System)

Under Jenkins global configuration, you have all the configurations of installed



Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel.: 91-0141-3500300 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

plugins and native Jenkins global configurations.

Also, you can configure global environment variables under this section. For example, you can store the tools (Nexus, Sonarqube, etc) URLs as global environment variables and use them in the pipeline. This way it is easier to make URL changes that get reflected in all the Jenkins jobs.

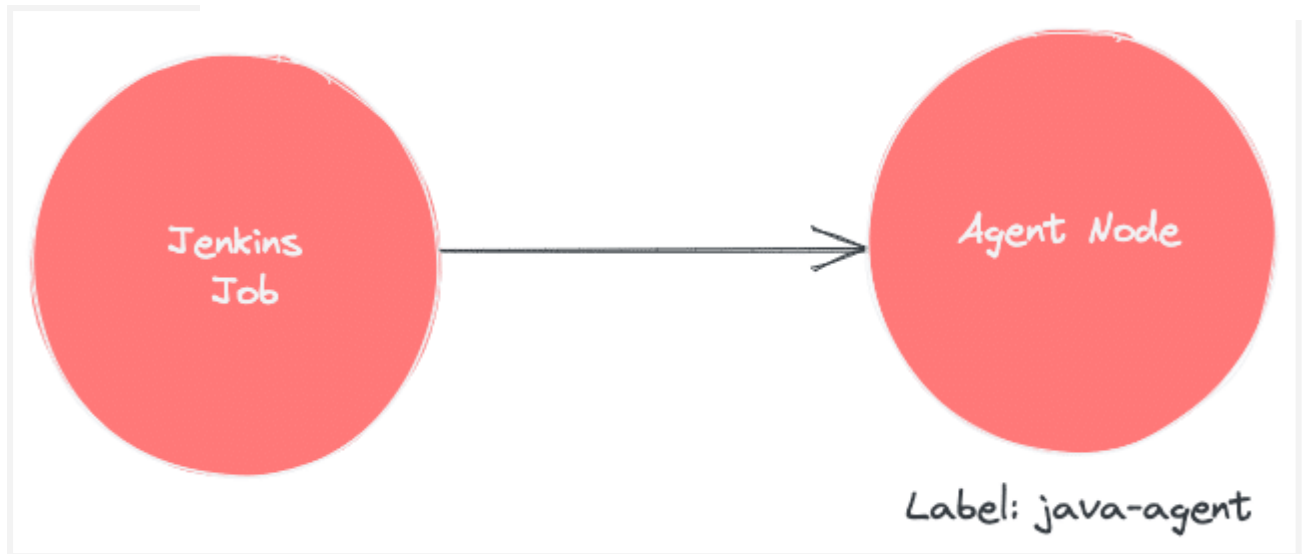
Jenkins Logs

Provides logging information on all Jenkins server actions including job logs, plugin logs, webhook logs, etc.

Note: All the configurations for the above-mentioned components are present as a config file (XML file) in the Jenkins master nodes data directory.

Jenkins Agent

Jenkins agents are the worker nodes that actually execute all the steps mentioned in a Job. When you create a Jenkins job, you have to assign an agent to it. Every agent has a label as a unique identifier.



When you trigger a Jenkins job from the master, the actual execution happens on the agent node that is configured in the job.

Note: You can run jobs in the Jenkins server without a Jenkins agent. In this case, master nodes acts as the agent. However, the recommended approach is to have a **Jenkins master-agent setup** for different job requirements so that you don't end up corrupting the Jenkins server for any system-wide configuration changes required for a job.

You can have any number of Jenkins agents attached to a master with a combination of Windows, Linux servers, and even containers as build agents.

Also, you can restrict jobs to run on specific agents, depending on the use case. For example, if you have an agent with java 8 configurations, you can assign this agent for jobs that require Java 8 environment.

There is no single standard for using the agents. You can set up a workflow and



strategy based on your project needs.

Jenkins server-agent Connectivity

You can connect a Jenkins master and agent in two ways

1. **Using the SSH method:** Uses the ssh protocol to connect to the agent. The connection gets initiated from the Jenkins master. There should be connectivity over port 22 between master and agent.
2. **Using the JNLP method:** Uses java JNLP protocol (**Java Network Launch Protocol**). In this method, a java agent gets initiated from the agent with Jenkins master details. For this, the master nodes firewall should allow connectivity on specified JNLP port. Typically the port assigned will be 50000. This value is configurable.

There are two types of Jenkins agents

1. **Agent Nodes:** These are servers (Windows/Linux) that will be **configured as static agents**. These agents will be up and running all the time and stay connected to the Jenkins server. Organizations use custom scripts to shut down and restart the agents when is not used. Typically during nights & weekends.
2. **Agent Clouds:** Jenkins Cloud agent is a concept of having **dynamic agents**. Means, whenever you trigger a job, a agent gets deployed as a VM/container on demand and gets deleted once the job is completed. This method saves money in terms of infra cost when you have a huge Jenkins ecosystem and



**Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA**

(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)

Approved by AICTE, Ministry of HRD, Government of India

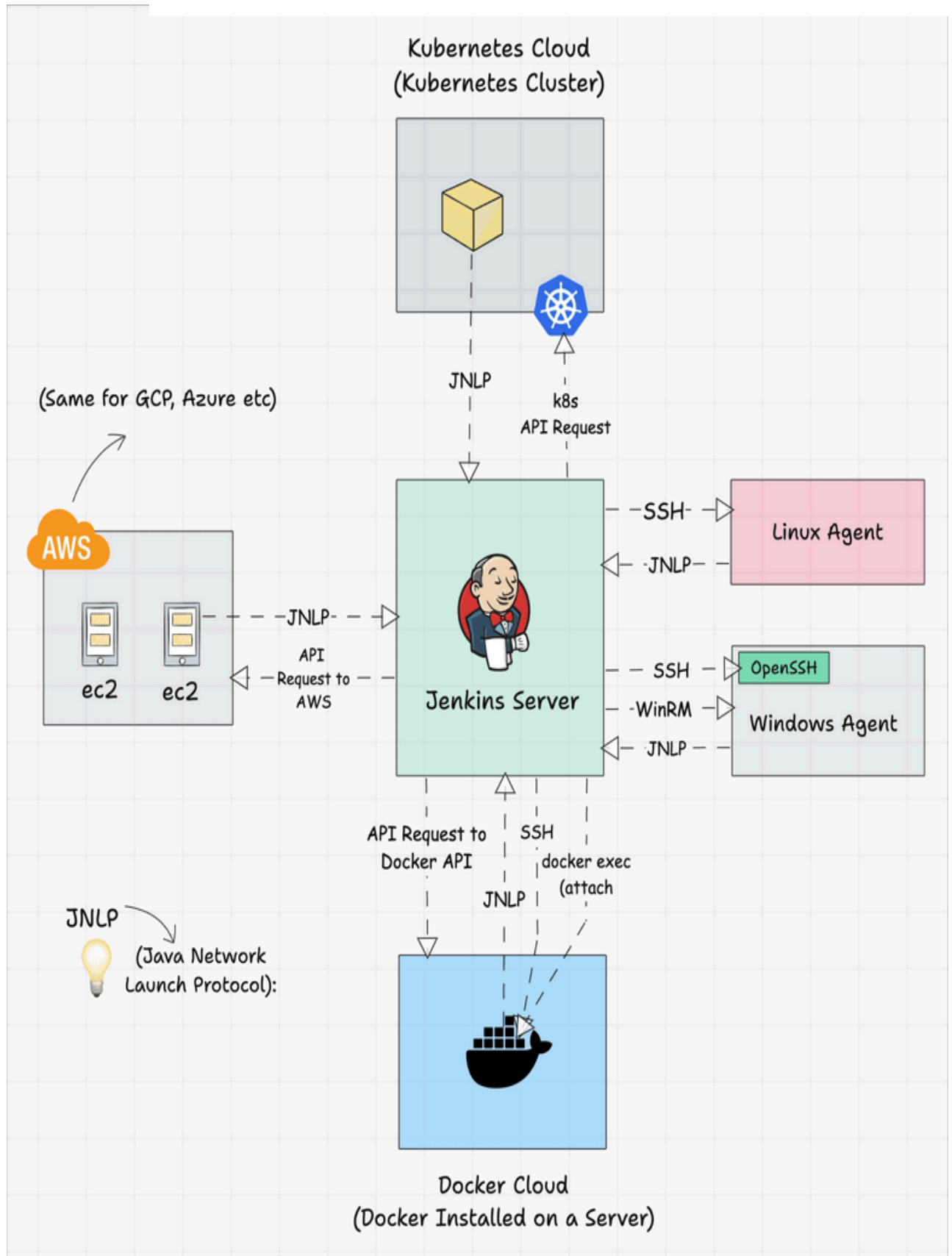
Recognized by UGC under Section 2(f) of the UGC Act, 1956

Tel.: 91-0141-3500300 Fax: +91-0141-2759555

E-mail: info@skit.ac.in Web: www.skit.ac.in

continuous basis.

The following image shows a high-level view of different types of agents and connectivity types.





Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagar, Jagatpura, Jaipur-302017, INDIA
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel.: 91-0141-3500300 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

You can refer to the following tutorials to understand more about Jenkins clouds.

1. [Docker container as build agents.](#)
2. [Setup Jenkins Build Agents on Kubernetes Pods](#)

Jenkins Data

All the Jenkins data gets stored in the following folder location.

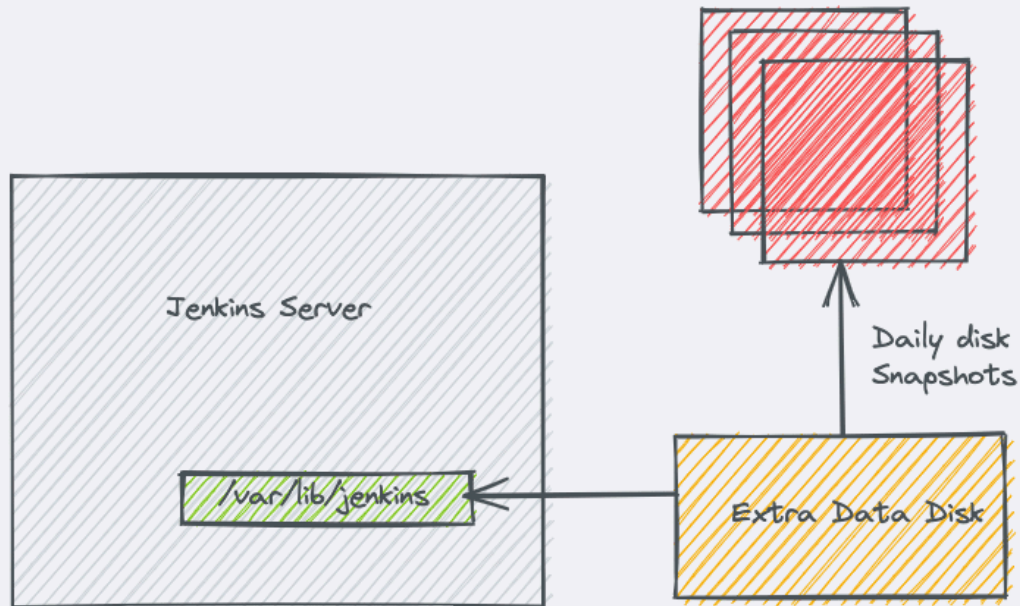
`/var/lib/jenkins/`

Data includes all jobs config files, plugins configs, secrets, node information, etc. It makes Jenkins migration very easy as compared to other tools.

If you take a look at `/var/lib/jenkins/` you will find most of the configurations in xml format.

It is essential to [back up the Jenkins data](#) folder every day. For some reason, if your Jenkins server data gets corrupt, you can restore the whole Jenkins with the data backup.

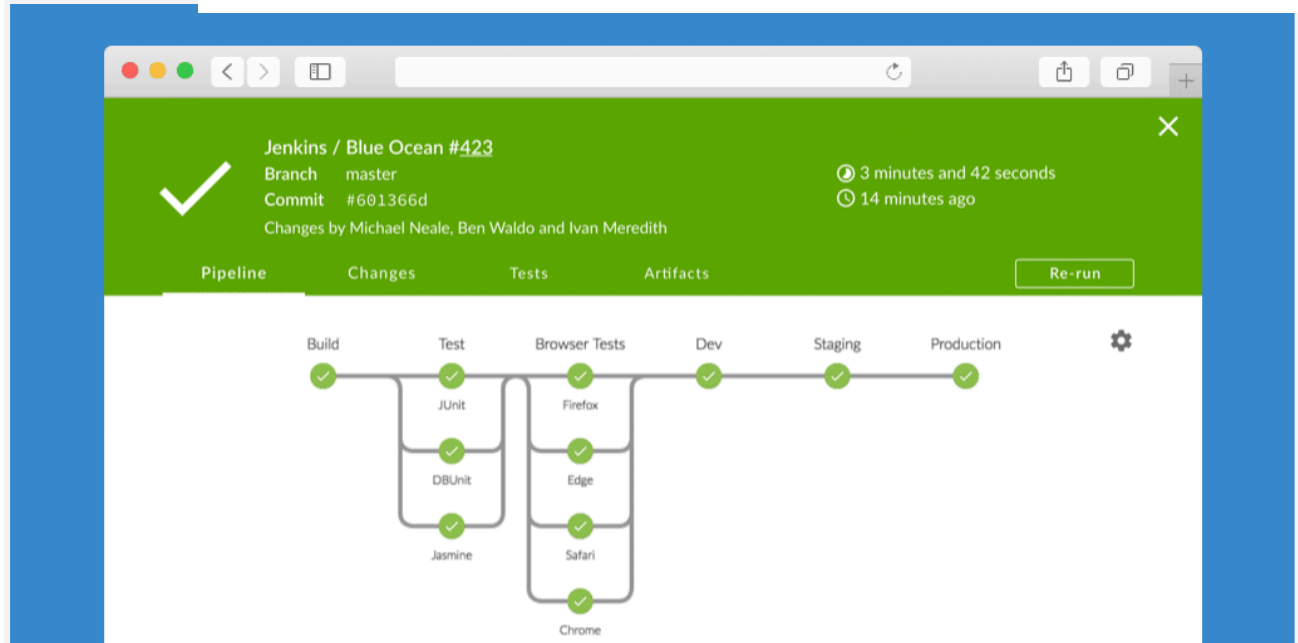
Ideally, when deploying Jenkins in production, a dedicated **extra volume is attached** to the Jenkins servers that hold all the Jenkins data.



Jenkins Web Interface

Jenkins 2.0 introduced a very intuitive web interface called "**Jenkins Blue Ocean**".

It has a good visual representation of all the pipelines.



Practical Jenkins Lab Activity: Build a CI/CD Pipeline Using Jenkins

Objective

To understand and implement a basic **CI/CD pipeline using Jenkins** that automatically fetches source code from GitHub, builds the project, executes tests, and generates a build result.

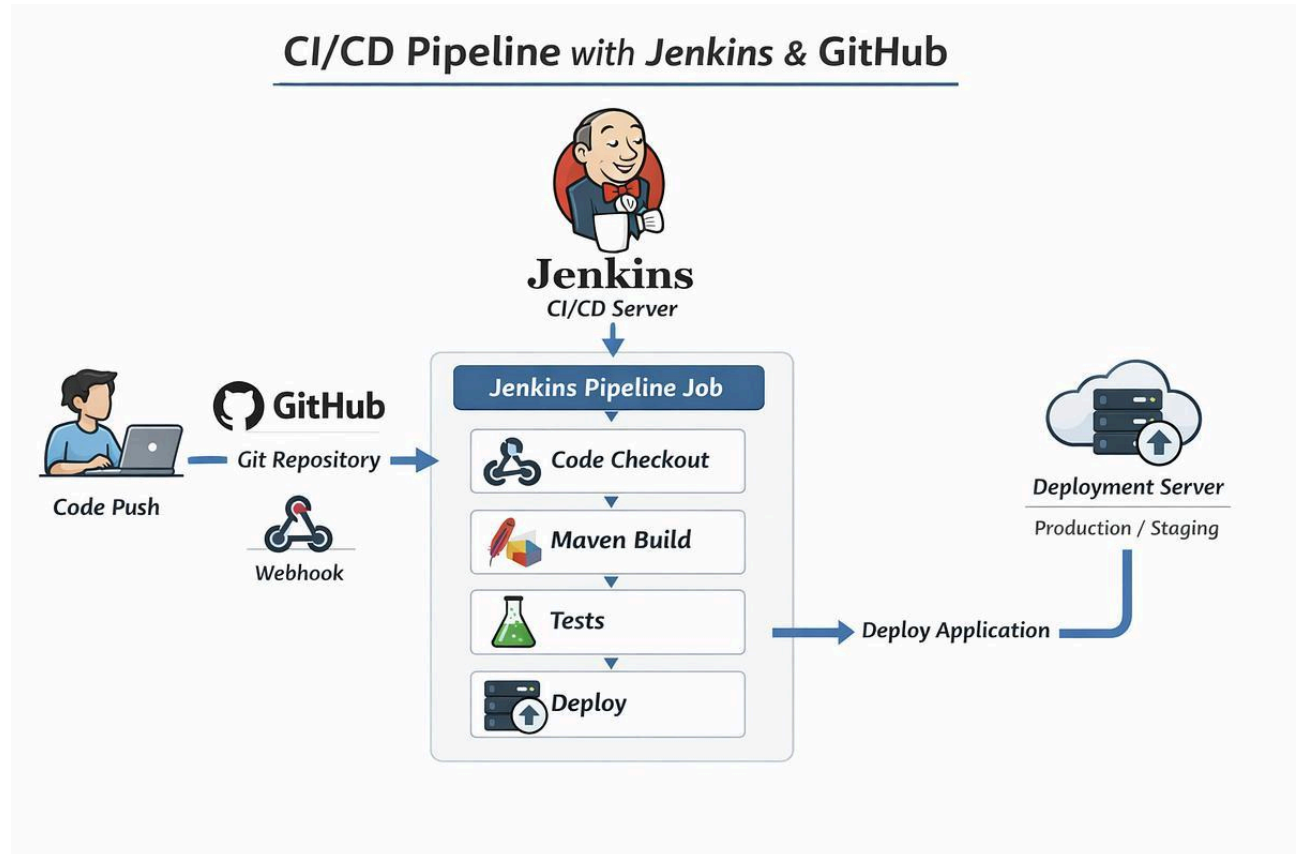
Prerequisites

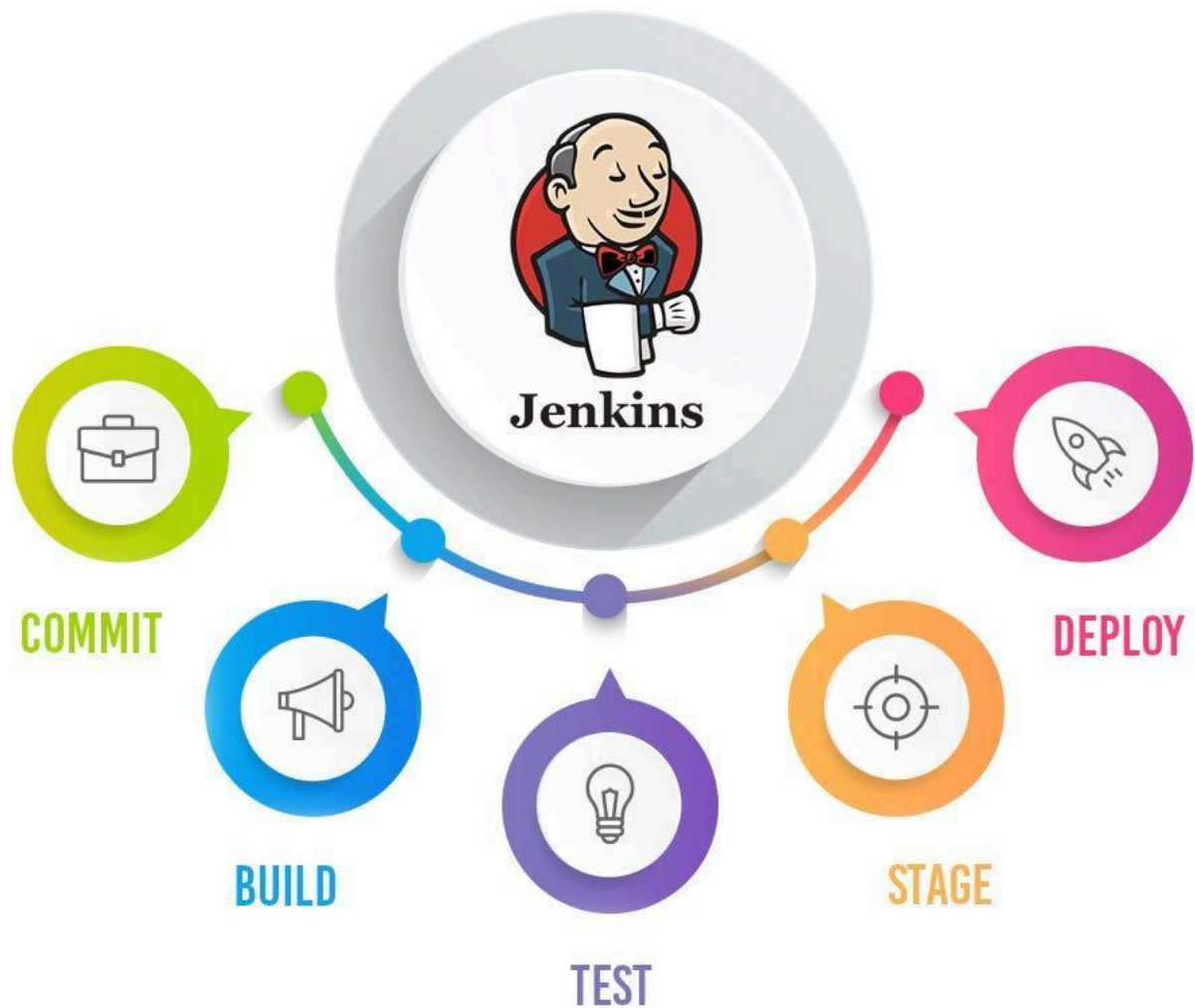
- Basic knowledge of Git and GitHub
- Jenkins installed and running
- Java JDK installed
- Git installed
- A GitHub account
- Web browser

Lab Scenario

You are a developer working on a Java application. Whenever a developer pushes updated code to GitHub, Jenkins should automatically:

GitHub → Jenkins → Checkout Code → Build → Test → Build Result







Jenkins

Jenkins > Hello World

Back to Dashboard

Status

Changes

Build Now

Delete Pipeline

Configure

Full Stage View

Rename

Pipeline Syntax

Build History trend

find

#1 11-May-2018 10:17

RSS for all RSS for failures

Pipeline Hello World

Recent Changes

Stage View

Average stage times:
(Average full run time: ~7s)

	Build	Test	Deploy
Average	308ms	173ms	153ms
#1	308ms	173ms	153ms

May 11 12:17 No Changes

Permalinks

- Last build (#1), 4 min 24 sec ago
- Last stable build (#1), 4 min 24 sec ago
- Last successful build (#1), 4 min 24 sec ago
- Last completed build (#1), 4 min 24 sec ago

6

Part 1: Create a GitHub Repository

1. Log in to GitHub.
2. Create a new repository named:

`jenkins-ci-demo`

3. Create a simple Java program:

```
public class HelloJenkins {  
    public static void main(String[] args) {  
        System.out.println("Hello from Jenkins CI Pipeline!");  
    }  
}
```

4. Save it as:

`HelloJenkins.java`



5. Commit and push the file to the repository.

Part 2: Verify Jenkins

Open Jenkins in your browser:

<http://localhost:8080>

Log in to the Jenkins dashboard.

Verify that Jenkins is running successfully.

Student Task

Record:

- Jenkins version
- Java version
- Git version
- Jenkins URL

Part 3: Create a Jenkins Freestyle Project

1. Click **New Item**.
2. Enter:

[Java-CI-Demo](#)

3. Select **Freestyle project**.
4. Click **OK**.

Configure Source Code Management

Select:

Git

Enter the URL of your GitHub repository.

Example:

<https://github.com/<username>/jenkins-ci-demo.git>



Select the appropriate branch, usually.

`*/main`

Part 4: Configure Build

Go to:

Build Steps → **Execute Windows batch command**

For Windows:

```
javac HelloJenkins.java  
java HelloJenkins
```

For Linux/macOS:

```
javac HelloJenkins.java  
java HelloJenkins
```

Click **Save**.

Part 5: Execute the Jenkins Build

1. Open the **Java-CI-Demo** project.
2. Click **Build Now**.
3. Observe the build number.
4. Click the build.
5. Open **Console Output**.

You should see:

```
Hello from Jenkins CI Pipeline!  
Finished: SUCCESS
```

Expected Result

The Jenkins job should complete successfully and display:

Finished: SUCCESS



Part 6: Introduce an Error

Now modify your Java program intentionally.

For example:

```
public class HelloJenkins {  
    public static void main(String[] args) {  
        System.out.println("Hello from Jenkins CI Pipeline!")  
    }  
}
```

Notice the missing semicolon.

Push the modified code to GitHub.

Run the Jenkins job again.

Observe

The build should fail.

Jenkins should display:

Finished: FAILURE

Student Task

Answer:

1. Why did the build fail?
 2. Where can you find the compilation error?
 3. How does Jenkins communicate build failure?
 4. How would you fix the problem?
-

Part 7: Fix the Error

Correct the program:

```
public class HelloJenkins {  
    public static void main(String[] args) {  
        System.out.println("Hello from Jenkins CI Pipeline!");  
    }  
}
```



Commit and push the changes.

Run the Jenkins job again.

Expected result:

Finished: SUCCESS

Part 8: Create a Jenkins Pipeline

Now create the same workflow using a **Pipeline job**.

Create a new Jenkins item:

Java-Pipeline-Demo

Select:

Pipeline

Go to:

Pipeline → Definition → Pipeline script

Enter:

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        echo 'Checking out source code'
      }
    }

    stage('Build') {
      steps {
        echo 'Building Java application'
        bat 'javac HelloJenkins.java'
      }
    }

    stage('Run') {
      steps {
        echo 'Running application'
```



```
        bat 'java HelloJenkins'
    }
}

post {
    success {
        echo 'Build completed successfully!'
    }

    failure {
        echo 'Build failed!'
    }
}
```

For Linux/macOS agents, replace **bat** with **sh**.

Click **Save** → **Build Now**.

Part 9: Create a Jenkinsfile

Instead of writing the pipeline directly inside Jenkins, create a file named:

Jenkinsfile

Add:

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                bat 'javac HelloJenkins.java'
            }
        }
    }
}
```



```
stage('Run') {  
    steps {  
        bat 'java HelloJenkins'  
    }  
}  
  
post {  
    success {  
        echo 'CI Pipeline completed successfully'  
    }  
  
    failure {  
        echo 'CI Pipeline failed'  
    }  
}
```

Push the Jenkinsfile to GitHub.

Then configure Jenkins to use:

Pipeline script from SCM → Git

Enter your GitHub repository URL.

Set:

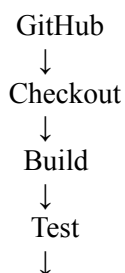
Script Path:

Jenkinsfile

Run the pipeline.

Part 10: Observe the Pipeline

Students should observe the following stages:





Deploy

For this basic lab, the implemented stages are:

Checkout → Build → Run

Practical Questions

Q1. What is Jenkins?

Q2. What is CI?

Q3. What is CD?

Q4. What is the difference between Freestyle Project and Pipeline?

Q5. What is a Jenkinsfile?

Q6. What is the purpose of the `stage()` block?

Q7. What does `agent any` mean?

Q8. What happens when a Jenkins build fails?

Q9. Why is GitHub integrated with Jenkins?

Q10. What is the difference between manual build and automated build?

Student Deliverables

Students must submit:

1. Screenshot of Jenkins Dashboard
2. Screenshot of GitHub repository
3. Screenshot of successful build
4. Screenshot of failed build
5. Screenshot of Console Output
6. Jenkins Pipeline screenshot
7. `Jenkinsfile`
8. Answers to practical questions



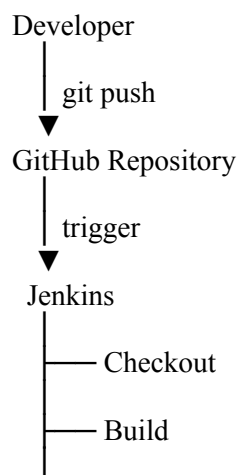
Evaluation / Viva

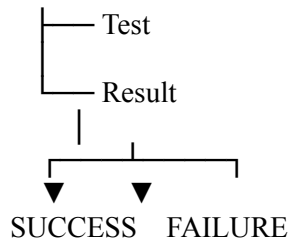
Activity	Marks
GitHub Repository Creation	2
Jenkins Job Configuration	2
Successful Build	2
Error Identification & Correction	2
Jenkins Pipeline/Jenkinsfile	2
Total	10

Viva Questions

- Why do we use Jenkins?
- What is continuous integration?
- What is a Jenkins job?
- What is a Jenkins pipeline?
- What is a Jenkinsfile?
- What is the role of GitHub in this experiment?
- What happens after a developer pushes code?
- How does Jenkins identify a failed build?
- What is the difference between **Build** and **Deploy**?
- What are Jenkins stages?

Final Expected Workflow





Extension Task: Configure a GitHub webhook so that a **git push** automatically triggers the Jenkins pipeline instead of using **Build Now** manually.

DevOps Theory Questions and Answers

Question 1

A software company is facing delays in software development and frequent production failures. How can DevOps help solve these issues?

Answer

DevOps is a software development methodology that combines Development (Dev) and Operations (Ops) to improve collaboration, automate workflows, and deliver high-quality software quickly and reliably. It focuses on continuous integration, continuous delivery, automation, monitoring, and communication between teams.

When a software company experiences delays in development and frequent production failures, DevOps provides several solutions that improve the entire software development lifecycle.

1. Improved Collaboration

Traditionally, developers and operations teams work separately, leading to communication gaps and misunderstandings. DevOps encourages collaboration throughout the development process, ensuring that both teams share responsibilities and work toward common goals.

Benefits:

- * Faster communication
- * Better coordination
- * Reduced misunderstandings
- * Faster issue resolution



2. Continuous Integration (CI)

Continuous Integration allows developers to frequently merge their code into a shared repository. Every code commit automatically triggers the build and testing process.

****Benefits:****

- * Detects bugs early
- * Prevents integration conflicts
- * Improves code quality
- * Reduces development delays

3. Continuous Delivery and Continuous Deployment (CD)

Continuous Delivery automates the release process so that software is always ready for deployment. Continuous Deployment automatically deploys verified code into production without manual intervention.

****Benefits:****

- * Faster software releases
- * Reduced deployment errors
- * Quick delivery of new features
- * Improved customer experience

4. Automated Testing

Manual testing consumes significant time and may fail to detect all defects. DevOps automates testing using tools that perform unit testing, integration testing, and security testing whenever new code is added.

****Benefits:****

- * Early bug detection
- * Reduced testing time
- * Higher software quality
- * Increased confidence before deployment

5. Infrastructure as Code (IaC)

Infrastructure as Code manages servers and cloud infrastructure using code instead of manual configuration. Tools like Terraform and AWS CloudFormation automate infrastructure provisioning.

****Benefits:****

- * Consistent environments



- * Reduced human errors
- * Faster infrastructure setup
- * Easy scalability

6. Continuous Monitoring

Monitoring tools continuously track application performance, server health, and system logs after deployment.

****Benefits:****

- * Early detection of failures
- * Quick recovery
- * Improved system availability
- * Better user experience

7. Automation

Automation eliminates repetitive manual tasks such as building, testing, deployment, and configuration.

****Benefits:****

- * Faster development cycle
- * Fewer manual mistakes
- * Increased productivity
- * Lower operational costs

Conclusion

By adopting DevOps practices such as Continuous Integration, Continuous Delivery, automated testing, Infrastructure as Code, continuous monitoring, and automation, the company can significantly reduce software development delays and production failures. DevOps enables faster software delivery, higher reliability, improved collaboration, and greater customer satisfaction.

Question 2

Design a DevOps workflow for an online banking application from development to deployment.

Answer

An online banking application requires high security, reliability, and availability. DevOps provides an automated workflow that ensures secure, fast, and reliable software delivery.



DevOps Workflow

Step 1: Requirement Analysis

Business analysts gather customer requirements such as online transactions, account management, loan services, and fund transfers.

Step 2: Code Development

Developers write application code using IDEs like Visual Studio Code or IntelliJ IDEA.

Programming languages may include:

- * Java
- * Python
- * Spring Boot
- * React

Step 3: Version Control

The source code is stored in Git repositories such as GitHub or GitLab.

Benefits:

- * Code version tracking
- * Team collaboration
- * Easy rollback
- * Branch management

Step 4: Continuous Integration (CI)

Whenever developers push code to GitHub, Jenkins automatically:

- * Downloads the latest code
- * Builds the application
- * Runs automated tests
- * Generates reports

This ensures that only working code proceeds further.

Step 5: Automated Testing



Different testing stages include:

- * Unit Testing
- * Integration Testing
- * Functional Testing
- * Security Testing
- * Performance Testing

Only successfully tested applications move to deployment.

Step 6: Build Process

Build tools like Maven or Gradle compile the code and generate executable files.

Outputs:

- * JAR files
- * WAR files
- * Docker images

Step 7: Containerization

Docker packages the application along with all required dependencies into containers.

Advantages:

- * Consistent execution
- * Easy deployment
- * Platform independence

Step 8: Deployment

Kubernetes manages container deployment.

Deployment process:

- * Development Environment
- * Testing Environment
- * Staging Environment
- * Production Environment

Deployment strategies include:



- * Blue-Green Deployment
- * Rolling Deployment

Step 9: Monitoring

Tools like Prometheus and Grafana monitor:

- * CPU usage
- * Memory usage
- * Network traffic
- * Application logs
- * Response time

Step 10: Feedback and Improvement

Operations teams provide performance reports and customer feedback to developers, enabling continuous improvement.

Workflow Diagram

Requirement Analysis → Development → Git Repository → Jenkins (CI) → Automated Testing → Build → Docker → Kubernetes Deployment → Monitoring → Feedback

Conclusion

A DevOps workflow automates every stage from development to deployment, ensuring secure, reliable, and faster delivery of online banking applications with minimal downtime.

Question 3

A team wants to automate software development and deployment. Suggest a suitable DevOps pipeline and tools.

Answer

A DevOps pipeline is a sequence of automated stages that enable software to move efficiently from development to production with minimal manual effort.

DevOps Pipeline



1. Planning

The project requirements are collected and tasks are assigned.

****Tools:****

- * Jira
- * Trello

2. Code Development

Developers write application code.

****Tools:****

- * Visual Studio Code
- * IntelliJ IDEA

3. Version Control

The code is stored and managed using Git repositories.

****Tools:****

- * Git
- * GitHub
- * GitLab

4. Continuous Integration

Whenever code is committed, Jenkins automatically:

- * Builds the application
- * Executes tests
- * Reports build status

****Tool:****

- * Jenkins

5. Build



The application is compiled into executable files.

****Tools:****

- * Maven
- * Gradle

6. Automated Testing

Testing ensures software quality.

Types:

- * Unit Testing
- * Integration Testing
- * Security Testing
- * Performance Testing

****Tools:****

- * JUnit
- * Selenium

7. Containerization

Applications are packaged into Docker containers.

****Tool:****

- * Docker

8. Deployment

Applications are deployed automatically using Kubernetes.

****Tool:****

- * Kubernetes

9. Configuration Management



Server configurations are automated and maintained.

****Tool:****

- * Ansible

10. Infrastructure as Code

Cloud infrastructure is created automatically.

****Tools:****

- * Terraform

- * AWS CloudFormation

11. Monitoring

Application performance is continuously monitored.

****Tools:****

- * Prometheus

- * Grafana

Complete DevOps Pipeline

Planning → Coding → Git → Jenkins → Build → Testing → Docker → Kubernetes → Monitoring → Feedback

Advantages

- * Faster software delivery

- * Reduced manual effort

- * Improved collaboration

- * Better software quality

- * Consistent deployments

- * Easy rollback

- * Reduced production failures

Conclusion



A DevOps pipeline integrates development, testing, deployment, and monitoring into one automated workflow, enabling organizations to release software quickly, efficiently, and reliably.

Question 4

Explain how Continuous Integration (CI) can reduce software defects with a real-world example.

Answer

Continuous Integration (CI) is a DevOps practice in which developers frequently merge their code into a shared repository. Every code change automatically triggers a build and testing process. This helps identify errors early and prevents defective software from reaching production.

How Continuous Integration Reduces Software Defects

1. Frequent Code Integration

Developers merge their work multiple times a day, reducing merge conflicts.

2. Automated Build Process

Every code commit is automatically compiled to verify that the application builds successfully.

3. Automated Testing

CI automatically runs:

- * Unit Tests
- * Integration Tests
- * Regression Tests

Any failed test immediately alerts developers.

4. Early Bug Detection

Defects are detected immediately after code changes rather than during final testing.



5. Better Code Quality

Developers fix issues while the code is still fresh, improving software quality.

Real-World Example

Consider an ****Amazon E-commerce Application**** where different developers work on login, payment, product search, and shopping cart modules.

Without Continuous Integration:

- * Developers merge code after several days.
- * Integration conflicts occur frequently.
- * Bugs remain unnoticed until final testing.
- * Releases are delayed.

With Continuous Integration:

- * Developers push code daily to GitHub.
- * Jenkins automatically builds the application.
- * Automated tests verify every feature.
- * If the payment module breaks the login system, Jenkins immediately reports the issue.
- * Developers fix the problem before deployment.

As a result:

- * Software defects are reduced.
- * Stable code reaches production.
- * Customers experience fewer issues.
- * Development becomes faster and more reliable.

Conclusion

Continuous Integration improves software quality by detecting defects early, reducing integration problems, automating testing, and ensuring that only verified code reaches production.

Question 5

Describe how DevOps practices improve customer satisfaction and business productivity.

Answer

DevOps combines development and operations through automation, collaboration, and continuous



improvement. It enables organizations to deliver software faster while maintaining high quality and reliability.

Improving Customer Satisfaction

1. Faster Feature Delivery

Customers receive new features and updates more frequently.

2. Better Software Quality

Automated testing identifies defects before deployment, reducing software failures.

3. Reduced Downtime

Automated deployment and monitoring minimize service interruptions.

4. Faster Bug Resolution

Continuous monitoring quickly identifies problems, allowing developers to fix issues rapidly.

5. Improved Performance

Regular monitoring and optimization ensure applications remain fast and responsive.

Improving Business Productivity

1. Automation

Automation reduces manual work in building, testing, deployment, and infrastructure management.

2. Increased Deployment Frequency

Organizations can release software multiple times a day instead of waiting for monthly releases.

3. Better Team Collaboration



Development and Operations teams work together, improving communication and reducing delays.

4. Lower Operational Costs

Automation reduces human effort, minimizes errors, and lowers maintenance expenses.

5. Faster Recovery

If failures occur, monitoring and rollback mechanisms help restore services quickly.

6. Higher Employee Productivity

Developers spend more time building new features rather than performing repetitive manual tasks.

Overall Benefits of DevOps

- * Faster software delivery
- * Improved collaboration
- * Higher software quality
- * Reduced production failures
- * Better security
- * Increased customer satisfaction
- * Greater business productivity
- * Faster recovery from failures
- * Lower operational costs

Conclusion

DevOps improves customer satisfaction by delivering reliable software quickly and resolving issues faster. At the same time, it enhances business productivity through automation, collaboration, continuous monitoring, and efficient deployment processes, enabling organizations to remain competitive in today's fast-paced software industry.